Rockchip Linux PCIe 问题排查指南

文件标识：RK-KF-YF-C63

发布版本：V1.1.0

日期：2026-04-16

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

产品版本

芯片名称	内核版本
所有芯片	所有SDK版本

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	林涛	2026-03-05	初始版本
V1.1.0	林鼎强	2026-04-16	补充 PCIe 外设 cache 一致性处理说明；优化 training 失败排查步骤，增加调试属性说明

目录

Rockchip Linux PCIe 问题排查指南

1. 常见应用问题
    1.1 硬件相关问题
        1.1.1 当走线位置不佳时，不同lane之间能否交织？
        1.1.2 同一个lane的差分信号能否交织？
        1.1.3 同一个PCIe接口是否支持拆分或者合并？
        1.1.4 PCIe 3.0接口支持哪些时钟输入模式？
        1.1.5 是否支持PCIe switch？贵司有没有推荐？
    1.2 软件相关问题
        1.2.1 在系统中如何确定控制器与设备的对应关系？
        1.2.2 如何确定PCIe设备的链路状态？
        1.2.3 是否支持Legacy INT方式？如何强制使用Legacy INTA ~ INTD的中断？
        1.2.4 如果CPU运行在32位地址模式下，如何实现BAR空间的访问？
        1.2.5 如何查看芯片分配给外设的CPU域地址以及PCIe bus域地址，两者如何对应？
        1.2.6 如何对下游单个设备进行重扫描或者在线更换设备？
        1.2.7 PCIe外设及其function驱动如何处理cache一致性？
        1.2.8 如何通过命令从RK PCIe EP发起MSI中断？
        1.2.9 如何通过命令从RK PCIe EP发起MSI-X中断？
        1.2.10 如何修改增加32bits-np映射地址空间？
        1.2.11 如何配置max payload size？
        1.2.12 如何固定被枚举设备的ID号？
    1.3 测试相关问题
        1.3.1 如何进入 PCIe 测试模式？
    1.4 功耗相关问题
        1.4.1 是否支持PCIe设备使用beacon方式唤醒主控？
        1.4.2 如何使得PCIe在二级休眠过程中保持工作状态
2. 异常排查
    2.1 链路建立问题
        2.1.1 驱动加载失败
        2.1.2 training 失败
        2.1.3 PCIe3.0控制器初始化设备系统异常
        2.1.4 PCIe2.0控制器初始化设备系统异常
    2.2 设备枚举和使用问题
        2.2.1 PCIe外设Memory BAR资源分配异常
        2.2.2 PCIe外设IO BAR资源分配异常
        2.2.3 无法读取配给外设的IO地址空间
        2.2.4 PCIe 设备可以Link却无法枚举出设备
        2.2.5 PCIe 设备可以枚举但访问异常
        2.2.6 外设枚举后通信过程中报错
        2.2.7 外设枚举过程报FW异常
        2.2.8 重新映射后访问PCIe设备的BAR地址空间异常
        2.2.9 PCIe转USB设备驱动(xhci)加载异常
    2.3 中断问题
        2.3.1 设备分配到legacy中断号为0
        2.3.2 MSI/MSI-X无法使用
    2.4 休眠唤醒问题
        2.4.1 PCIe 3.0接口休眠唤醒时系统异常
        2.4.2 PCIe设备切换PM模式时模块异常
        2.4.3 PCIe 外设休眠唤醒时模块无法连接或者工作异常
    2.5 稳定性问题
        2.5.1 NVMe 设备长期工作状态下出现异常
    2.6 其他问题
        2.6.1 PCIe转SATA设备盘号不固定
        2.6.2 GKI模式下PCIe外设工作异常
3. 附录

3.1 LTSSM状态机
3.2 PCIe地址空间配置详述
3.3 M.2接口硬件

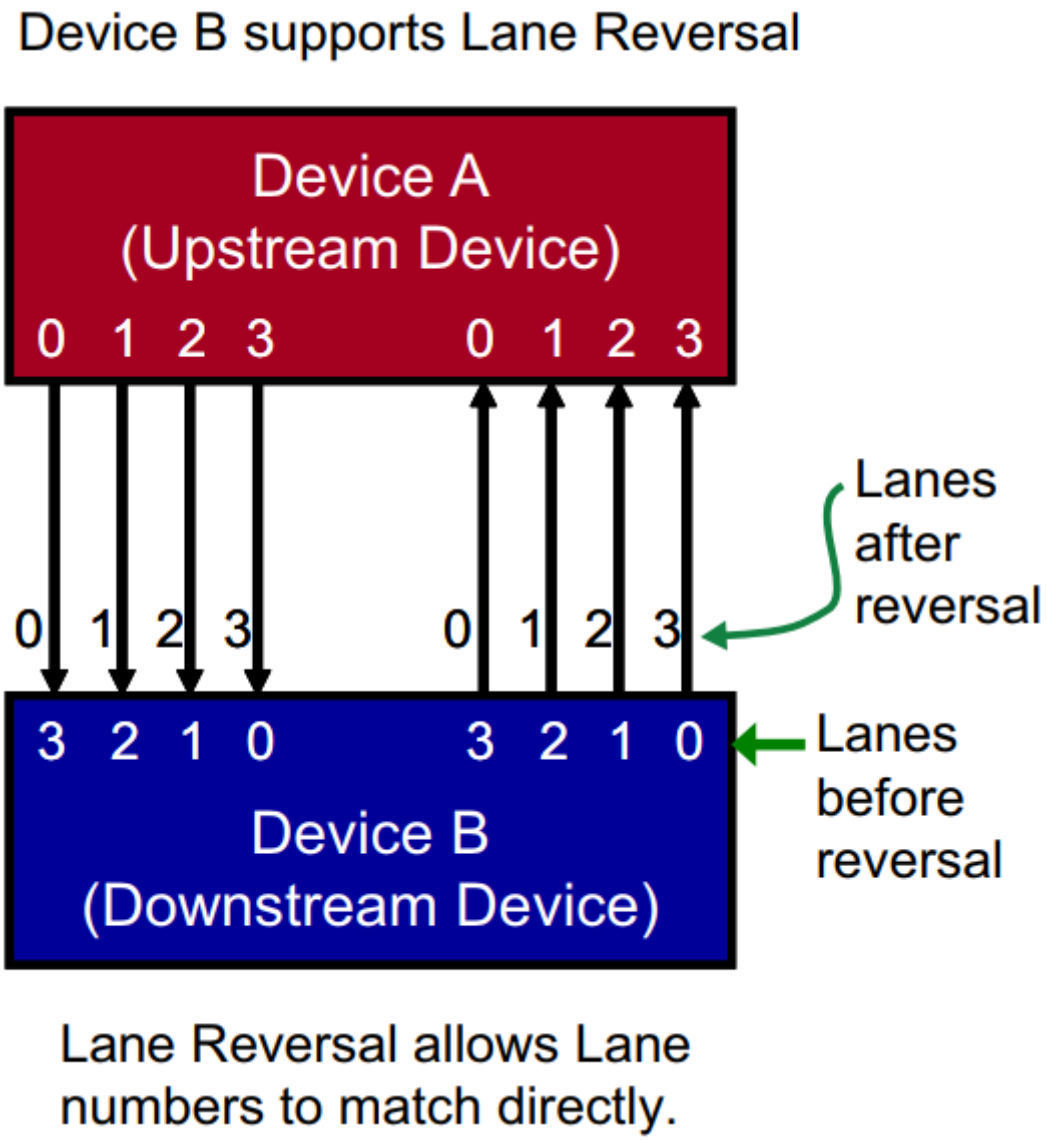
1. 常见应用问题

1.1 硬件相关问题

1.1.1 当走线位置不佳时，不同lane之间能否交织？

1. 支持lane交织（Lane reversal），属于硬件协议行为，软件不需要改动。但是有如下限制：

- 4 Lane的情况下，目前RK平台仅支持全倒序交织，即RC的Lane[0.1.2.3] 分别对应 EP的Lane[3.2.1.0]，其余情况一概不支持。



- 2 Lane的情况下，同理支持RC的Lane[0.1]分别对应EP的Lane[1.0]。

2. 不支持同一侧不同组lane之间的信号的组合使用，如RC lane0 TX与lane1 RX无法组合为一组lane来外接设备。

1.1.2 同一个lane的差分信号能否交织？

支持PN翻转（Lane polarity），通常RC TX+接入EP RX+、RC TX-接入EP RX-，支持RC TX+接入EP RX-等PN翻转布线，且软件不需要额外处理，由PCIe控制器自动探测。

不支持TX/RX交织，例如RC的lane1 TX无法与EP的lane1TX对接。

1.1.3 同一个PCIe接口是否支持拆分或者合并？

RK芯片的部分PCIe口可以支持拆分和合并功能，请参阅"芯片资源介绍"章节，具体方法参考示例和dts说明。

1.1.4 PCIe 3.0接口支持哪些时钟输入模式？

PCIe 3.0 的PHY的输入时钟模式可以是HCSL、LPHCSL或者其他差分信号，例如支持LVDS外加电平转换等电路实现的输入时钟方案，以上所有以满足PHY指标为准。

Power Supplies:

vph Min = 1.8 V-10%; Typ = 1.8 V; Max = 1.8 V+10% or
vph Min = 1.5 V-5%; Typ = 1.5 V; Max = 1.5 V+10% or
vph Min = 1.2 V-5%; Typ = 1.2 V; Max = 1.2 V+10%
Note: 1.2 V support is pending silicon characterization and validation

vp/vpdig/vptxX Min = 0.9 V-7%; Typ = 0.9 V; Max = 0.9 V+10%

Symbol	Parameter	Min	Max	Units	Conditions
FREF_CLK	Reference clock frequency	19.2	200	MHz	Not continuous; per protocol as noted in the databook
FREF_OFFSET	Reference clock frequency offset	-100	100	ppm	All protocols except for SATA
RMSJREF_CLK <= 8 Gbps	Reference clock Random Jitter		1.6	ps_{rms}	Integrated RJ from 12 kHz to 20 MHz
			1.2		Integrated Rj from 2 MHz to 20 MHz
DJREF_CLK <= 8 Gbps	Reference clock deterministic jitter		2.8	ps, pp	0.75-10 MHz (offset)[a]
			5.6		0.2-50 MHz (offset)[a]
DCREF_CLK	Duty cycle	40	60	%	

Symbol	Parameter	Min	Max	Units	Conditions
VCMREF_CLk	Common mode input level	0	vp	V	Differential inputs
VDREF_CLK	Differential input swing	300	2*vp	mVppd	Differential input must meet both VDREF_CLK and VDSEREF_CLK
VDSEREF_CLK	Differential clock single ended voltage	0	vp		Differential input must meet both VDREF_CLK and VDSEREF_CLK
SKEWDIF_CLK	Differential clock input skew		100	ps	
SWREF_CLK	Differential clock input slew rate	0.1		V/ns	20 – 80%

Note:
For PCIe, follow the CEM specification requirements

a. DJ noise limits are for noise within an offset from the divided reference clock and harmonics of the divided reference clock

1.1.5 是否支持PCIe switch？贵司有没有推荐？

理论上支持，不需要任何补丁，且没有推荐列表。为了把控风险，请联系供应商借评估板，插在我司EVB上验证后再采购。

1.2 软件相关问题

1.2.1 在系统中如何确定控制器与设备的对应关系？

以RK3568芯片为例：

PCIe2x1控制器给外设分配的Bus地址介于0x0~0xf，PCIe3x1控制器给外设分配的bus地址介于0x10~0x1f，PCIe3x2控制器给外设分配的bus地址介于0x20~0x2f。从lspci输出的信息中可以看到各设备分配到的bus地址（高位），即可确定对应关系。第二列Class是设备类型，第三列VID:PID。Class类型请参考https://pci-ids.ucw.cz/read/PD/，厂商VID和产品PID请参考 http://pci-ids.ucw.cz/v2.2/pci.ids

```
console:/ # lspci
21:00.0 Class 0108: 144d:a808
20:00.0 Class 0604: 1d87:3566
11:00.0 Class 0c03: 1912:0014
10:00.0 Class 0604: 1d87:3566
01:00.0 Class 0c03: 1912:0014
00:00.0 Class 0604: 1d87:3566
```

我们可以看到每个控制器下游预留了16级bus来接设备，意味着每个控制器下游可以接16个设备(含switch), 一般可以满足需求，阅读者可以跳过下面的说明。如果确属需要调整，请调整rk3568.dtsi中三个控制器的bus-range分配，且务必确保不要重叠。另外，调整bus-range将导致设备的MSI(-X) RID区间变化，请同步调整msi-map。

```
bus-range = <起始地址    结束地址>

msi-map = < bus-range中的起始地址 << 8
            &its
            bus-range中的起始地址 << 8
            bus-range中分配的总线总数 << 8>
```

例如bus-range调整为0x30 ~ 0x5f, 即该控制器下游设备分配的bus地址从0x30 到0x5f, 总线总数 0x30个

则可配置 msi-map = <0x3000 &its 0x3000 0x3000>

依此类推，且一定要保证三个控制器的bus-range和msi-map互不重叠，且bus-range和msi-map相互适配。

1.2.2 如何确定PCIe设备的链路状态？

请使用服务器发布的lspci工具，执行lspci -vvv，找到对应设备的linkStat即可查看；其中Speed为速度，Width即为lane数。如需要解析其他信息，请查找搜索引擎，对照查看。

1.2.3 是否支持Legacy INT方式？如何强制使用Legacy INTA ~ INTD的中断？

支持legacy INT方式。但Linux PCIe协议栈默认的优先级是MSI-X, MSI, Legacy INT，因此常规市售设备不会去申请Legacy INT。若调试测试需要，请参考内核中Documentation/admin-guide/kernel-parameters.txt文档，其中"pci=option[,option...] [PCI] various PCI subsystem options."描述了可以在cmdline中关闭MSI，则系统默认会强制使用Legacy INT分配机制。以RK356X安卓平台为例，可在arch/arm64/boot/dts/rockchip/rk3568-android.dtsi的cmdline参数中额外添加一项pci=nomsi，注意前后项需空格隔开：

```
bootargs = "...... pci=nomsi ......";
```

如果添加成功，则lspci -vvv可以看到此设备的MSI和MSI-X都是处于关闭状态(Enable-)，而分配了INT A中断，中断号是80。cat /proc/interrupts可查看到80中断的状态。

```
01:00.0 Class 0108: Device 14a4:22f1 (rev 01) (prog-if 02)
        Subsystem: Device 1b4b:1093
...
        Interrupt: pin A routed to IRQ 80
...
        Capabilities: [50] MSI: Enable- Count=1/1 Maskable+ 64bit+
                Address: 0000000000000000  Data: 0000
                Masking: 00000000  Pending: 00000000
...
        Capabilities: [b0] MSI-X: Enable- Count=19 Masked-
                Vector table: BAR=0 offset=00002000
                PBA: BAR=0 offset=00003000
```

1.2.4 如果CPU运行在32位地址模式下，如何实现BAR空间的访问？

默认状态下，芯片给PCIe控制器分配的BAR空间均位于超出32位寻址的地址段。但是我们芯片有预留32位以下的一个BAR地址，对每个控制器有不同的限制。例如，RK3568芯片每个控制器的低位BAR地址仅有32MB，当RK3568的CPU运行在32位地址模式下，此时我们应该启用低位地址空间对每个PCIe节点的ranges进行重新分配。以下例子已经修改为配置空间1MB，IO空间1MB和32-bit MEM空间30MB：

```
&pcie2x1 {
        ranges = <0x00000800 0x0 0xF4000000 0x0 0xF4000000 0x0 0x100000
                  0x81000000 0x0 0xF4100000 0x0 0xF4100000 0x0 0x100000
                  0x82000000 0x0 0xF4200000 0x0 0xF4200000 0x0 0x1e00000>;
}

&pcie3x1 {
        ranges = <0x00000800 0x0 0xF2000000 0x0 0xF2000000 0x0 0x100000
                  0x81000000 0x0 0xF2100000 0x0 0xF2100000 0x0 0x100000
                  0x82000000 0x0 0xF2200000 0x0 0xF2200000 0x0 0x1e00000>;
}

&pcie3x2 {
        ranges = <0x00000800 0x0 0xF0000000 0x0 0xF0000000 0x0 0x100000
                  0x81000000 0x0 0xF0100000 0x0 0xF0100000 0x0 0x100000
                  0x82000000 0x0 0xF0200000 0x0 0xF0200000 0x0 0x1e00000>;
}
```

如需调整各个空间的大小，可参考附录中“关于PCIe地址空间配置详述”的部分。

1.2.5 如何查看芯片分配给外设的CPU域地址以及PCIe bus域地址，两者如何对应？

使用lspci命令可以看到各设备CPU域地址以及PCIe bus域地址

```
root@linaro-alip: /home/linaro# lspci -Vs 0002:21:00.0 -X
0002:21 :00.0 Non-Volatile memory controller: Intel Corporation SSD Pro 7600p/
760p/E 6100p Series (rev 03) (prog-if 02 [NVM Express] )
    Subsystem: Intel Corporation SSD Pro 7600p/ 760p/E 6100p Series
    Flags: bus master, fast devsel, latency 0, IRQ 114
    Memory at 380900000 (64-bit, non-prefetchable) [size=16K]
    Capabilities: [40] Power Management version 3
    ....

00: 86 80 a6 f1 06 04 10 00 03 02 08 01 00 00 00 00
10: 04 00 90 80 00 00 00 00 00 00 00 00 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 86 80 0b 39
30: 00 00 00 00 40 00 00 00 00 00 00 00 72 01 00 00
```

我们可以看出到，芯片给外设分配的CPU域地址是0x380900000；CPU若需要访问外设，可以直接用IO命令或者devmem命令读取0x380900000。而0x380900000这个CPU域地址对应的PCIe bus域地址，可从PCIe外设的BAR0地址(偏移0x10开始到0x14为止)读出“04 00 90 80”，即为0x80900000（最低位04是表明此BAR的类型为64bit MEM）。因此当CPU发出访问0x380900000这个CPU域地址时，PCIe的地址转换服务(ATU)，通过所配置的outbound关系，可以发出0x80900000这个PCIe bus域地址，从而实现CPU访问到PCIe外设的内部信息。

而两者的对应关系在rk3568.dtsi中有定义，我们以pcie3x2为例：

```
ranges = <0x00000800 0x0 0x80000000 0x3 0x80000000 0x0 0x800000
          0x81000000 0x0 0x80800000 0x3 0x80800000 0x0 0x100000   IO空间
          0xc3000000 0x0 0x80900000 0x3 0x80900000 0x0 0x3f700000>; Memory空间
```

例如我们可查看到Memory段的详细分配情况。首段0xc3000000表明此memory段为64-bit prefetchable空间。
0x3 0x80900000为芯片分配的CPU域地址，即0x0000000380900000；0x0 0x80900000为分配的CPU域地址所对应的PCIe bus域地址，即0x0000000080900000；0x3f700000为此memory段的总大小。
因此CPU发出的CPU域访问地址与转换后的PCIe bus域地址存在0x300000000的偏移关系，这部分偏移关系在硬件上由ATU自动转化。

关于地址空间的配置详情，可参考附录中“关于PCIe地址空间配置详述”的部分。

1.2.6 如何对下游单个设备进行重扫描或者在线更换设备？

若有如下两点需求则需对下游设备进行重枚举。

1. 下游设备可能在不同阶段出现bar空间变化；
2. 下游设备损坏后进行在线更换；

```
(1) 找出下游所需更换或者重扫描设备，目前我们以BDF为01:00.0的这个设备为例
console:/ # /data/lspci -k
00:00.0 Class 0604: Device 1d87:3566 (rev 01)
        Kernel driver in use: pcieport
01:00.0 Class 0c03: Device 1912:0014 (rev 03)

(2) 对设备进行remove操作，可看到对应设备节点以及它所运行的pcie driver被反初始化
console:/ # echo 1 > /sys/bus/pci/devices/0000\:01\:00.0/remove
[   30.624938] xhci_hcd 0000:01:00.0: remove, state 4
[   30.624985] usb usb8: USB disconnect, device number 1
[   30.625741] xhci_hcd 0000:01:00.0: USB bus 8 deregistered
[   30.626115] xhci_hcd 0000:01:00.0: remove, state 4
[   30.626142] usb usb7: USB disconnect, device number 1
[   30.640977] xhci_hcd 0000:01:00.0: USB bus 7 deregistered
[   32.055886] vcc5v0_otg: disabling
[   32.055920] vcc3v3_lcd1_n: disabling

(3) 可再次查看，确定设备已经无法扫描到了
console:/ # /data/lspci -k
00:00.0 Class 0604: Device 1d87:3566 (rev 01)
        Kernel driver in use: pcieport

(4) 发起总线重扫描，可任选如下两条指令之一，执行后新设备恢复识别
console:/ #  echo 1 > /sys/bus/pci/devices/0000\:00\:00.0/rescan
console:/ #  echo 1 > /sys/bus/pci/rescan
[   33.222240] pci 0000:01:00.0: BAR 0: assigned [mem 0x300900000-0x300900fff
64bit]
[   33.222606] xhci_hcd 0000:01:00.0: xHCI Host Controller
[   33.224875] xhci_hcd 0000:01:00.0: new USB bus registered, assigned bus
number 7
[   33.225468] xhci_hcd 0000:01:00.0: hcc params 0x002841eb hci version 0x100
quirks 0x0000000000000090
[   33.226318] usb usb7: New USB device found, idVendor=1d6b, idProduct=0002,
bcdDevice= 4.19
```

```
[   33.226329] usb usb7: New USB device strings: Mfr=3, Product=2,
SerialNumber=1
[   33.226345] usb usb7: Product: xHCI Host Controller
[   33.226355] usb usb7: Manufacturer: Linux 4.19.172 xhci-hcd
[   33.226364] usb usb7: SerialNumber: 0000:01:00.0
[   33.227661] hub 7-0:1.0: USB hub found
[   33.227716] hub 7-0:1.0: 1 port detected
[   33.228252] xhci_hcd 0000:01:00.0: xHCI Host Controller
[   33.228581] xhci_hcd 0000:01:00.0: new USB bus registered, assigned bus
number 8
[   33.226816] xhci_hcd 0000:01:00.0: Host supports USB 3.0 SuperSpeed
[   33.228678] usb usb8: We don't know the algorithms for LPM for this host,
disabling LPM.
[   33.228783] usb usb8: New USB device found, idVendor=1d6b, idProduct=0003,
bcdDevice= 4.19
[   33.228796] usb usb8: New USB device strings: Mfr=3, Product=2,
SerialNumber=1
[   33.228803] usb usb8: Product: xHCI Host Controller
[   33.228809] usb usb8: Manufacturer: Linux 4.19.172 xhci-hcd
[   33.228814] usb usb8: SerialNumber: 0000:01:00.0
[   33.229360] hub 8-0:1.0: USB hub found
[   33.229406] hub 8-0:1.0: 4 ports detected
[   33.556216] usb 7-1: new high-speed USB device number 2 using xhci_hcd
[   33.700886] usb 7-1: New USB device found, idVendor=2109, idProduct=3431,
bcdDevice= 4.20
[   33.700913] usb 7-1: New USB device strings: Mfr=0, Product=1, SerialNumber=0
[   33.700920] usb 7-1: Product: USB2.0 Hub
[   33.702398] hub 7-1:1.0: USB hub found
[   33.702642] hub 7-1:1.0: 4 ports detected
```

1.2.7 PCIe外设及其function驱动如何处理cache一致性？

根据PCIe协议Snoop和No Snoop的定义，外设向主机系统进行访问请求，其TLP包需要设置No Snoop属性位，来表征是否需要主机系统的硬件协助其完成cache一致性的维护。

No Snoop 位	Cache Coherency 类型	管理模型	备注
0	Snoop	硬件保证cache一致性	Cfg包，I/O包，Message包，MSI(-x)包必须设置为0
1	No Snoop	外设驱动维护cache一致性	-

当外设发出写入主机内存的TLP包，若其No Snoop位为0，进入主机系统后，RC将检查所写入的地址是否落入其他CPU的cache内。若是，则需要在数据写入物理内存后，对所有含有此地址缓存的CPU执行invalidate操作。

当外设发来从主机内存取数据并送往外设的TLP包，若其No Snoop位为0，则RC将检查所有CPU的cache，确定该TLP包内数据所在的地址是否有被其他CPU缓存。若是，则在取数据之前进行flush操作。

RK平台的部分芯片体系并不支持Snoop类型，统一按照No Snoop为1处理。因此运行的function驱动需要对非DMA coherent方式申请的内存自行维护cache一致性。支持Snoop类型的，芯片可以保证cache一致性。

需注意算力卡、显卡此类外设：此类设备因长期应用于Windows环境，通常默认依赖RC平台的Snoop机制保证数据一致性；若移植到不支持Snoop的平台，务必确认是否已完成一致性兼容处理。

1.2.8 如何通过命令从RK PCIe EP发起MSI中断？

测试前确认：

- RC和EP都支持并使能MSI cap，RK RC和RK EP默认已使能MSI cap
- RC 上运行EP的function驱动，并已申请msi中断

RK PCIe EP client寄存器PCIE_CLIENT_MSI_GEN_CON 支持触发MSI中断，32bits寄存器分别对应32个msi 中断，以RK3588为例：

```
io -4 0xFE150038 1
```

除此之外，还可通过：

- 设置outbound atu，CPU写发起memory write
- 或通过DMA传输发起memory write

其他说明：

- 部分PC只能申请一个MSI中断，例如包括Linux 系统和Windows系统，主要受PC BIOS配置限制，例如：
  - Intel BIOS虚拟化及VT-d配置
  - Interrupt remapper support
- RK3588 RC上能申请32个MSI中断

1.2.9 如何通过命令从RK PCIe EP发起MSI-X中断？

原理

PC通过标准的MSI-X cap映射BAR4配置MSI table

RK3568实例

```
# 加载驱动后msix配置后状态，从设备端执行以下两条命令可以看到MSI-Xtable
io -4 0xfe280270 0x10001       # 测试时开启 Dbi writeable
io -4 -l 0x100 0xf6300000
io -4 0xfe280270 0x10000

f6300000:  fee02004 00000000 00000024 00000000
f6300010:  fee08004 00000000 00000024 00000000
f6300020:  fee01004 00000000 00000024 00000000
f6300030:  fee02004 00000000 00000025 00000000
f6300040:  fee08004 00000000 00000025 00000000
f6300050:  fee01004 00000000 00000025 00000000
f6300060:  fee02004 00000000 00000026 00000000
f6300070:  fee04004 00000000 00000026 00000000
f6300080:  00000000 00000000 00000000 00000001
f6300090:  00000000 00000000 00000000 00000001
f63000a0:  00000000 00000000 00000000 00000001
f63000b0:  00000000 00000000 00000000 00000001
f63000c0:  00000000 00000000 00000000 00000001
```

```
f63000d0:  00000000 00000000 00000000 00000001
f63000e0:  00000000 00000000 00000000 00000001
f63000f0:  00000000 00000000 00000000 00000001

# 由于当前业务没有提供 MSI-X 接口示例，可通过以下方式触发 MSI-X
# 设置设备 outbound，默认RK EP outbound 配置 CPU addr 为 0xf0000000，测试时未改动，或者通过内核接口完成 Bar outbound 配置
io -4 0xf6300014 0xfee00000     # 指向前面的 io -4 -l 0x100 0xf6300000 输出地址
io -4 0xf6300018 0x0

# 触发 MSI-X
io -4 0xf0002004 0x24
io -4 0xf0008004 0x24
io -4 0xf0001004 0x24
io -4 0xf0002004 0x25
io -4 0xf0008004 0x25
io -4 0xf0001004 0x25
io -4 0xf0002004 0x26
io -4 0xf0004004 0x26
```

RK3588实例

```
# 加载驱动后 msix 配置后状态，从设备端执行以下两条命令可以看到 MSI-X table
io -4 0xfe150270 0x10001        # 测试时开启 Dbi writeable
io -4 -l 0x100 0xf5300000
io -4 0xfe150270 0x10000

f5300000:  fee02004 00000000 00000024 00000000
f5300010:  fee08004 00000000 00000024 00000000
f5300020:  fee01004 00000000 00000024 00000000
f5300030:  fee02004 00000000 00000025 00000000
f5300040:  fee08004 00000000 00000025 00000000
f5300050:  fee01004 00000000 00000025 00000000
f5300060:  fee02004 00000000 00000026 00000000
f5300070:  fee04004 00000000 00000026 00000000
f5300080:  00000000 00000000 00000000 00000001
f5300090:  00000000 00000000 00000000 00000001
f53000a0:  00000000 00000000 00000000 00000001
f53000b0:  00000000 00000000 00000000 00000001
f53000c0:  00000000 00000000 00000000 00000001
f53000d0:  00000000 00000000 00000000 00000001
f53000e0:  00000000 00000000 00000000 00000001
f53000f0:  00000000 00000000 00000000 00000001

# 由于当前业务没有提供 MSI-X 接口示例，可通过以下方式触发 MSI-X
# 设置设备 outbound，默认RK EP outbound 配置 CPU addr 为 0xf0000000，测试时未改动，或者通过内核接口完成 Bar outbound 配置
io -4 0xf5300014 0xfee00000     # 指向前面的 io -4 -l 0x100 0xf5300000 输出地址
io -4 0xf5300018 0x0

# 触发 MSI-X
io -4 0xf0002004 0x24
io -4 0xf0008004 0x24
io -4 0xf0001004 0x24
io -4 0xf0002004 0x25
io -4 0xf0008004 0x25
io -4 0xf0001004 0x25
```

```
io -4 0xf0002004 0x26
io -4 0xf0004004 0x26
```

1.2.10 如何修改增加32bits-np映射地址空间？

背景

PCIe mmio空间为CPU发起PCIe传输的映射地址空间，物理地址专用，Dram不包括此段空间，每个PCIe控制器有专用的mmio空间，通常为32MB或者16MB 32bits地址空间和1GB 64bits地址空间。

PCIe range为CPU访问地址和PCI域虚拟地址之间的映射，PCIe range分配的原则：

- 域地址与Dram物理地址不能重叠，避免EP端DMA地址混乱导致访问错误，所以：
  - 默认域地址分配在对应PCIe控制器mmio物理地址，与PCIe mmio空间一一对应
  - 扩展少量32bits-np域地址空间，建议跨用其他PCIe控制器32bits-np域地址；
- 扩展32bit-np域地址所需的物理mmio地址只能从本控制器的64bits 物理mmio地址去借用，不能使用其他PCIe控制器的物理mmio地址。
- 继续扩充更多32bits-np域地址空间，建议先查询TRM手册System Map章节是否有大块的其他IP的32bits的物理mmio地址可用。若有，则借用这块地址来充当32bits-np域地址；若无则跨用reserved 4GB内memory空间做进一步分配

默认配置提供确认，以RK3588为例：

```
[    5.325570] pci_bus 0000:00: root bus resource [mem 0xf0200000-0xf0ffffff]
                       # RC 14MB 32bits-np mem
[    5.325577] pci_bus 0000:00: root bus resource [mem 0x900000000-0x93fffffff
pref]                   # RC 1GB 64bits-pref mem
```

问题：默认分配32bits-np空间不足

log：

```
[   11.646077] pci 0000:01:00.0: reg 0x10: [mem 0x00000000-0x00ffffff 64bit]   #
Dev总共需求26MB 32bits-np，最终会对齐需求32MB空间
[   11.646113] pci 0000:01:00.0: reg 0x18: [mem 0x00000000-0x007fffff 64bit]
[   11.646150] pci 0000:01:00.0: reg 0x20: [mem 0x00000000-0x001fffff 64bit]
...
[   11.971710] pci 0000:01:00.0: BAR 0: no space for [mem size 0x01000000 64bit]
[   11.971713] pci 0000:01:00.0: BAR 0: failed to assign [mem size 0x01000000
64bit]
[   11.971717] pci 0000:01:00.0: BAR 2: no space for [mem size 0x00800000 64bit]
[   11.971720] pci 0000:01:00.0: BAR 2: failed to assign [mem size 0x00800000
64bit]
[   11.971723] pci 0000:01:00.0: BAR 4: no space for [mem size 0x00200000 64bit]
[   11.971726] pci 0000:01:00.0: BAR 4: failed to assign [mem size 0x00200000
64bit]
```

解决补丁参考

参考 TRM 确认 PCIe 专用的内存空间，然后新增所缺mem range资源。

实例1：RK3588 pcie3x4修改240MB 32bits-np映射空间配置，修改内核rk3588.dtsi对应节点range属性为：

```
                ranges = <0x00000800 0x0 0xff000000 0x0 0xf0000000 0x0 0x100000
                          0x81000000 0x0 0xff100000 0x0 0xf0100000 0x0 0x100000
                          0x82000000 0x0 0xf0000000 0x9 0x00000000 0x0
0x0f000000
                          0xc3000000 0x9 0x10000000 0x9 0x10000000 0x0
0x30000000>;
```

实例2：RK3588s pcie2x1l2修改240MB 32bits-np映射空间配置，修改内核rk3588s.dtsi对应节点range属性为：

```
                ranges = <0x00000800 0x0 0xff000000 0x0 0xf4000000 0x0 0x100000
                          0x81000000 0x0 0xff100000 0x0 0xf4100000 0x0 0x100000
                          0x82000000 0x0 0xf0000000 0xa 0x00000000 0x0
0x0f000000
                          0xc3000000 0xa 0x10000000 0xa 0x10000000 0x0
0x30000000>;
```

实例3：RK3588 pcie2x1l0修改240MB 32bits-np映射空间配置，修改内核rk3588.dtsi对应节点range属性为：

```
                ranges = <0x00000800 0x0 0xff000000 0x0 0xff000000 0x0 0x100000
                          0x81000000 0x0 0xff100000 0x0 0xff100000 0x0 0x100000
                          0x82000000 0x0 0xf0000000 0x9 0x80000000 0x0
0x0f000000
                          0xc3000000 0x9 0x90000000 0x9 0x90000000 0x0
0x30000000>;
```

实例4：RK3568 pcie3x2修改240MB 32bits-np映射空间配置，修改内核rk356x.dtsi对应节点range属性为：

```
                ranges = <0x00000800 0x0 0xff000000 0x0 0xf0000000 0x0 0x100000
                          0x81000000 0x0 0xff100000 0x0 0xf0100000 0x0 0x100000
                          0x82000000 0x0 0xf0000000 0x3 0x80000000 0x0
0x0f000000
                          0xc3000000 0x3 0x90000000 0x3 0x90000000 0x0
0x30000000>;
```

实例5：基于实例1新增0x30000000处的256M映射内存，并做按照要求做内存预留：

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
index a10dad37f9cf..d1b16e13c459 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
@@ -233,6 +233,15 @@ wireless_wlan: wireless-wlan {
                WIFI,poweren_gpio = <&gpio3 RK_PB1 GPIO_ACTIVE_HIGH>;
                status = "okay";
        };
+
+       reserved-memory {
+               #address-cells = <2>;
+               #size-cells = <2>;
+               ranges;
+               pcie3x4_range: pcie3x4-range@30000000 {
+                       reg = <0x0 0xdfe00000 0x0 0x10200000>;
```

```
+                };
+        };
 };

 &backlight {
diff --git a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
index ad414c61fd38..096f16740e11 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
@@ -601,10 +601,11 @@ pcie3x4: pcie@fe150000 {
                 phys = <&pcie30phy>;
                 phy-names = "pcie-phy";
                 power-domains = <&power RK3588_PD_PCIE>;
-                ranges = <0x00000800 0x0 0xf0000000 0x0 0xf0000000 0x0 0x100000
-                          0x81000000 0x0 0xf0100000 0x0 0xf0100000 0x0 0x100000
-                          0x82000000 0x0 0xf0200000 0x0 0xf0200000 0x0 0xe00000
-                          0xc3000000 0x9 0x00000000 0x9 0x00000000 0x0
0x40000000>;
+                 ranges = <0x00000800 0x0 0xdfe00000 0x0 0xf0000000 0x0 0x100000
+                           0x81000000 0x0 0xdff00000 0x0 0xf0100000 0x0 0x100000
+                           0x82000000 0x0 0xe0000000 0x9 0x00000000 0x0
0x20000000 // 扩展至512MB
+                           0xc3000000 0x9 0x20000000 0x9 0x20000000 0x0
0x20000000>;
                 reg = <0x0 0xfe150000 0x0 0x10000>,
                       <0xa 0x40000000 0x0 0x400000>;
                 reg-names = "pcie-apb", "pcie-dbi";
```

1.2.11 如何配置max payload size？

PCIe以TLP的形式发送数据，而max payload size(简称mps)决定了pcie设备的tlp能够传输的最大字节数。mps的大小是由PCIe链路两端的设备协商决定的，PCIe设备发送TLP时，其最大payload不能超过mps的值。

内核提供基于dts的"[PCI] various PCI subsystem options."配置，详细参考“Documentation/admin-guide/kernel-parameters.txt”文档，相关配置如下：

```
pcie_bus_tune_off   #关闭mps调整，使用设备自身默认值
pcie_bus_safe       #开启mps调整，设置所有设备都支持的最大mps值
pcie_bus_perf       #开启mps调整，根据parent bus及自身capability设置为最大mps
pcie_bus_peer2peer #开启mps调整，设置所有设备为mps为128B
```

说明：

- 内核默认max payload size配置机制为pcie_bus_tune_off
- 通常可以考虑直接在dts bootargs添加pci=pcie_bus_safe属性

1.2.12 如何固定被枚举设备的ID号？

当系统中出现多路同类型的PCIe设备，如多个网卡，由于初始化顺序不固定的情况，所以eth0代表哪一路设备实际并不固定。同样的情况也适用于NVMe等。用户如果希望将被枚举设备的ID进行固定，需要修改对应的function驱动，修改的理论基础是设备的bus number是DTS固定分配的。以下两个例子可供参考：

```
如果RK3588接了三个网卡，想按下面顺序固定：
pcie2x1l0: pcie@fe170000 => eth1
pcie2x1l2: pcie@fe190000 => eth2
pcie2x1l1: pcie@fe180000 => eth3

查询dtsi可知：
pcie2x1l0节点：bus-range = <0x20 0x2f> -> 网卡分配的bus number为0x21 -> eth1
pcie2x1l2节点：bus-range = <0x40 0x4f> -> 网卡分配的bus number为0x41 -> eth2
pcie2x1l1节点：bus-range = <0x30 0x3f> -> 网卡分配的bus number为0x31 -> eth3

修改对应的function驱动，在register_netdev函数被调用前修改注册到网络子系统的名字。
--- a/drivers/net/ethernet/realtek/r8168/r8168_n.c
+++ b/drivers/net/ethernet/realtek/r8168/r8168_n.c
@@ -25481,6 +25481,18 @@ static const struct net_device_ops rtl8168_netdev_ops =
{
 };
 #endif

+static void pci_bus_nr_2_id(struct pci_dev *pdev, struct net_device *ndev )
+{
+
+       dev_info(&pdev->dev, "%s pdev->bus->number = 0x%x\n",
+                __func__, pdev->bus->number);
+
+       if(pdev->bus->number == 0x21)
+               strcpy(ndev->name, "eth1");
+       if(pdev->bus->number == 0x41)
+               strcpy(ndev->name, "eth2");
+       if(pdev->bus->number == 0x31)
+               strcpy(ndev->name, "eth3");
+}
+
 static int __devinit
 rtl8168_init_one(struct pci_dev *pdev,
                  const struct pci_device_id *ent)
@@ -25624,7 +25636,7 @@ rtl8168_init_one(struct pci_dev *pdev,
                 rtl8168_set_eeprom_sel_low(tp);

         rtl8168_get_mac_address(dev);
-
+       pci_bus_nr_2_id(pdev, dev); // 修改位置一定要在register_netdev()之前
         tp->fw_name = rtl_chip_fw_infos[tp->mcfg].fw_name;

同理NVMe存储器可参考类似修改
--- a/drivers/nvme/host/core.c
+++ b/drivers/nvme/host/core.c
@@ -5169,6 +5169,19 @@ static void nvme_free_ctrl(struct device *dev)
                 nvme_put_subsystem(subsys);
 }

+static void pci_bus_nr_2_id(struct pci_dev *pdev, struct nvme_ctrl *ctrl)
+{
```

```
+       dev_info(&pdev->dev, "%s pdev->bus->number = 0x%x\n",
+                __func__, pdev->bus->number);
+
+       if(pdev->bus->number == 0x21)
+               ctrl->instance = 1; //pcie2x1l0 -> nvme1
+       if(pdev->bus->number == 0x41)
+               ctrl->instance = 2; //pcie2x1l2 -> nvme2
+       if(pdev->bus->number == 0x31)
+               ctrl->instance = 3; //pcie2x1l1 -> nvme3
+}
+
 /*
  * Initialize a NVMe controller structures.  This needs to be called during
  * earliest initialization so that we have the initialized structured around
@@ -5178,6 +5191,7 @@ int nvme_init_ctrl(struct nvme_ctrl *ctrl, struct device
*dev,
                const struct nvme_ctrl_ops *ops, unsigned long quirks)
 {
        int ret;
+       struct pci_dev *pdev = container_of(dev, struct pci_dev, dev);

        ctrl->state = NVME_CTRL_NEW;
        clear_bit(NVME_CTRL_FAILFAST_EXPIRED, &ctrl->flags);
@@ -5214,6 +5228,8 @@ int nvme_init_ctrl(struct nvme_ctrl *ctrl, struct device
*dev,
                goto out;
        ctrl->instance = ret;

+       pci_bus_nr_2_id(pdev, ctrl);
        device_initialize(&ctrl->ctrl_device);
        ctrl->device = &ctrl->ctrl_device;
        ctrl->device->devt = MKDEV(MAJOR(nvme_ctrl_base_chr_devt),
```

1.3 测试相关问题

1.3.1 如何进入 PCIe 测试模式？

标准测试套件

参考`Rockchip_Developer_Guide_PCIe_CN`文档“控制器dts常用配置“章节中的rockchip,compliance-mode属性说明。

外设TX信号眼图

直接飞线测量TX信号结果，通常无法与标准测试套件的结果直接比较，只能用于同版型设备间的差异对比。信号指标应参考PCIe标准协议。软件无需特殊配置，设备枚举成功后即可进行带宽测试，如NVMe可用fio命令测试读写。

PCIe 3.0PHY参数软件不支持调整

PCIe Gen3为专用PHY，软件参数已经是最佳适配参数，不支持调整。

PCIe Gen3大部分TX/RX参数为PHY tuning结果，不支持调整。

PCIe 2.0PHY参数软件不建议调整

PCIe Gen1/2 PHY通常使用combophy，软件参数已经是最佳适配参数，通常不允许再次调整。

如发现信号不佳：

- 确认信号测试方法是否规范？
- 确认PCB符合RK EVB图纸规范？

如最终确认信号存在明显缺陷，请优化PCB硬件设计。

1.4 功耗相关问题

1.4.1 是否支持PCIe设备使用beacon方式唤醒主控？

RK平台不支持在L2的状态下，由PCIe设备发送beacon唤醒主控。如果有唤醒需求，请使用WAKEN信号当作GPIO唤醒源方式实现。

1.4.2 如何使得PCIe在二级休眠过程中保持工作状态

目前，PCIe的默认休眠唤醒策略是：

- 发送PME_Turn_Off包，使得双端都进入L2状态
- 停止参考时钟输出，关闭EP电源，双端进入D3 cold状态
- Trusted Firmware对相关总线和PLL进行断电
- 唤醒后重新扫描和枚举设备

由于部分设备，例如Wi-Fi，需要支持wake-up-host功能，需要在二级待机下保持其电源常开，所以其电源节点没有配置到PCIe节点中进行控制(vpcie3v3-supply)，而是由Wi-Fi funtion驱动进行控制。

如果除了电源以外，某些设备连时钟/链路都不允许停止，则需要：

1. 注释PCIe驱动的休眠唤醒函数

```
--- a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
+++ b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
@@ -1902,6 +1902,8 @@ static int __maybe_unused rockchip_dw_pcie_suspend(struct
device *dev)
        struct dw_pcie *pci = rk_pcie->pci;
        u32 status;

+       return 0;
+
        /*
         * This is as per PCI Express Base r5.0 r1.0 May 22-2019,
         * 5.2 Link State Power Management (Page #440).
@@ -1997,6 +1999,8 @@ static int __maybe_unused rockchip_dw_pcie_resume(struct
device *dev)
        struct dw_pcie *pci = rk_pcie->pci;
        int ret;

+       return 0;
```

2. 保持PCIe的PD常开，总线电源常开

```
# 1.查询dtsi中PCIe控制器节点所引用的power-domains节点
```

```
pcie0: pcie@2a200000 {
    ...
    power-domains = <&power RK3576_PD_PHP>;
    ...
};

# 2.将这个power-domain节点配置成常开，并修改休眠状态下的电源控制
--- a/arch/arm64/boot/dts/rockchip/rk3576.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3576.dtsi
@@ -1717,11 +1717,8 @@
rockchip_suspend: rockchip-suspend {
    compatible = "rockchip,pm-config";
    status = "disabled";

    rockchip,sleep-debug-en = <0>;
    rockchip,sleep-mode-config = <
            (0
-           | RKPM_SLP_ARMOFF_LOGOFF
-           | RKPM_SLP_PMU_PMUALIVE_32K
-           | RKPM_SLP_PMU_DIS_OSC
+           | RKPM_SLP_ARMOFF_DDRPD
            | RKPM_SLP_PMIC_LP
-           | RKPM_SLP_32K_EXT
            )
    >;

@@ -2325,7 +2325,7 @@
power-domain@RK3576_PD_PHP {
    reg = <RK3576_PD_PHP>;
    #address-cells = <1>;
    #size-cells = <0>;
    pm_qos = <&qos_mmu0>,
             <&qos_mmu1>;
-
+   rockchip,always-on;
    power-domain@RK3576_PD_SUBPHP {
        reg = <RK3576_PD_SUBPHP>;
    };
```

3. 联系底层平台中心负责Arm Trusted Firmware的相关技术人员，获取更新的bl31固件，可保持相关总线和PLL存活，更新后请重编U-boot固件。

2. 异常排查

2.1 链路建立问题

2.1.1 驱动加载失败

```
[    0.417008] rk-pcie 3c0000000.pcie: Linked as a consumer to regulator.14
[    0.417477] rk-pcie 3c0800000.pcie: Linked as a consumer to regulator.14
[    0.417648] rk-pcie 3c0800000.pcie: phy init failed
```

异常原因：dts中未正确开启此控制器所对应的phy节点。

```
[    0.195567] rochchip_p3phy_init: lock failed 0x6890000, check input refclk
and power supply
[    0.195585] phy phy-fe8c0000.phy.8: phy init failed --> -110
[    0.195599] rk-pcie 3c0800000.pcie: fail to init phy, err -110
[    0.195611] rk-pcie 3c0800000.pcie: phy init failed
```

异常原因：PCIe 3.0 PHY工作电源或者输入时钟异常，导致phy没有正常工作。

2.1.2 training 失败

```
PCIe Link Fail的log如下一直重复”PCIe Linking...“，LTSSM状态机可能不同

rk-pcie 3c0000000.pcie: PCIe Linking... LTSSM is 0x0
rk-pcie 3c0000000.pcie: PCIe Linking... LTSSM is 0x0
rk-pcie 3c0000000.pcie: PCIe Linking... LTSSM is 0x0

或者出现"PCIe Link up"的打印，但LTSSM状态机的bit16与bit17不等于0x3，bit0到bit8数值小于
0x11
[    3.108325] rk-pcie fe150000.pcie: Looking up vpcie3v3-supply from device
tree
[    3.126926] rk-pcie fe150000.pcie: missing legacy IRQ resource
[    3.126940] rk-pcie fe150000.pcie: IRQ msi not found
[    3.126944] rk-pcie fe150000.pcie: use outband MSI support
[    3.126947] rk-pcie fe150000.pcie: Missing *config* reg space
[    3.126954] rk-pcie fe150000.pcie: host bridge /pcie@fe150000 ranges:
[    3.126965] rk-pcie fe150000.pcie:      err 0x00f0000000..0x00f00fffff ->
0x00f0000000
[    3.126972] rk-pcie fe150000.pcie:       IO 0x00f0100000..0x00f01fffff ->
0x00f0100000
[    3.126979] rk-pcie fe150000.pcie:      MEM 0x00f0200000..0x00f0ffffff ->
0x00f0200000
[    3.126984] rk-pcie fe150000.pcie:      MEM 0x0900000000..0x090fffffff ->
0x0900000000
[    3.127007] rk-pcie fe150000.pcie: Missing *config* reg space
[    3.127028] rk-pcie fe150000.pcie: invalid resource
[    3.387304] rk-pcie fe150000.pcie: PCIe Link up, LTSSM is 0x0

真正如果link成功，应该可以看到类似log，出现"PCIe Link up"的打印，且LTSSM状态机的bit16与
bit17等于0x3，bit0到bit8数值大于等于0x11
[    2.410536] rk-pcie 3c0000000.pcie: PCIe Link up, LTSSM is 0x130011
```

异常原因：training 失败，外设没有处于工作状态或者信号异常，首先确认低速IO配置正确：

- PERSTN，对应dts reset-gpios属性，电平应存在高低变化
- CLKREQN，默认未使用，电平常低
- WAKEN，默认未使用，电平常高

其次，检测以下硬件信号质量（默认枚举仅持续2秒，枚举超时会关闭控制器，如需延长枚举时间可在dts节点添加 `rockchip,wait-for-link-ms = <10000>` 属性以方便信号测量）：

- 外设的3V3供电是否提供
- 电源功率是否足够，尤其是FPGA/算力卡等外设有较高的功率需求，部分外设还需要12V电源
- 电源质量是否良好，是否存在异常噪声

- PERSTN复位信号与电源上电时序是否与此设备的spec冲突

如果都无法解决，大概率需要定位信号完整性：

- 需要拿出测试眼图和PCB给到我司硬件，并且最好我们建议贵司找实验室提供一份测试TX/RX兼容性信号测试报告。

另外还建议客户打开pcie-dw-rockchip.c中的RK_PCIE_DBG，抓一份log以便分析。请阅读者注意，如果有多个控制器同时使用，抓log前请先把不使用或者没问题的设备对应的控制器disable掉，这样log会好分析一点。所抓取的log中，将会出现类似“rk-pcie 3c0000000.pcie: fifo_status = 0x144001”等信息。fifo_status的末尾两位是PCIe链路的ltssm状态机，可以根据状态机信息判断异常发生的大致情况。芯片的PCIe ltssm状态机信息可参照文末附录部分。

2.1.3 PCIe3.0控制器初始化设备系统异常

```
[   21.523506] rcu: INFO: rcu_preempt detected stalls on CPUs/tasks:
[   21.523557] rcu:     1-...0: (0 ticks this GP) idle=652/1/0x4000000000000000
softirq=30/30 fqs=2097
[   21.523579] rcu:     3-...0: (5 ticks this GP) idle=4fa/1/0x4000000000000000
softirq=35/36 fqs=2097
[   21.523590] rcu:     (detected by 2, t=6302 jiffies, g=-1151, q=98)
[   21.523610] Task dump for CPU 1:
[   21.523622] rk-pcie         R  running task        0    55      2 0x0000002a
[   21.523640] Call trace:
[   21.523666]  __switch_to+0xe0/0x128
[   21.523682]  0x43752cfcfe820900
[   21.523694] Task dump for CPU 3:
[   21.523704] kworker/u8:0    R  running task        0     7      2 0x0000002a
[   21.523737] Workqueue: events_unbound enable_ptr_key_workfn
[   21.523751] Call trace:
[   21.523767]  __switch_to+0xe0/0x128
[   21.523786]  event_xdp_redirect+0x8/0x90
[   21.523816] rcu: INFO: rcu_sched detected stalls on CPUs/tasks:
[   21.523840] rcu:     1-...0: (50 ticks this GP) idle=652/1/0x4000000000000000
softirq=7/30 fqs=2099
[   21.523859] rcu:     3-...0: (55 ticks this GP) idle=4fa/1/0x4000000000000000
softirq=5/36 fqs=2099
[   21.523870] rcu:     (detected by 2, t=6302 jiffies, g=-1183, q=1)
[   21.523887] Task dump for CPU 1:
[   21.523898] rk-pcie         R  running task        0    55      2 0x0000002a
[   21.523915] Call trace:
[   21.523931]  __switch_to+0xe0/0x128
[   21.523944]  0x43752cfcfe820900
[   21.523955] Task dump for CPU 3:
[   21.523965] kworker/u8:0    R  running task        0     7      2 0x0000002a
[   21.523990] Workqueue: events_unbound enable_ptr_key_workfn
[   21.524004] Call trace:
```

异常原因：如果系统卡住此log附近，则表明PCIe3.0的PHY工作异常。请依次检查

- 外部晶振芯片的时钟输入是否异常，如果无时钟或者幅度异常，将导致phy无法锁定。
- 检查 PCIE30_AVDD_0V9 和PCIE30_AVDD_1V8电压是否满足要求。
- 检查板级DTS中是否有将PCIe的低速IO进行配置，如有请删除再测试；并参考'低速IO说明'部分了解这些IO的具体使用方式。
- RK3588 pcie30phy 如果只使用其中一个port，另一个port也需要供电，检查是否符合要求。

2.1.4 PCIe2.0控制器初始化设备系统异常

```
[   21.523870] rcu:     (detected by 2, t=6302 jiffies, g=-1183, q=1)
[   21.523887] Task dump for CPU 1:
[   21.523898] rk-pcie         R  running task        0    55      2 0x0000002a
[   21.523915] Call trace:
[   21.523931]  __switch_to+0xe0/0x128
[   21.523944]  0x43752cfcfe820900
[   21.523955] Task dump for CPU 3:
[   21.523965] kworker/u8:0    R  running task        0     7      2 0x0000002a
[   21.523990] Workqueue: events_unbound enable_ptr_key_workfn
[   21.524004] Call trace:
```

异常原因：如果系统卡住此log附近，则表明PCIe2.0的PHY工作异常。以RK3568的combphy2_psq这个PHY为例，请依次检查

- 检查 PCIE30_AVDD_0V9 和PCIE30_AVDD_1V8电压是否满足要求。
- 检查板级DTS中是否有将PCIe的低速IO进行配置，如有请删除再测试；并参考'低速IO说明'部分了解这些IO的具体使用方式。
- 修改combphy2_psq的驱动phy-rockchip-naneng-combphy.c，在rockchip_combphy_init函数的末尾增加如下代码，检查PHY内部的一些配置：

```
val = readl(priv->mmio + (0x27 << 2));
dev_err(priv->dev, "TXPLL_LOCK is 0x%x PWON_PLL is 0x%x\n",
val & BIT(0), val & BIT(1));
val = readl(priv->mmio + (0x28 << 2));
dev_err(priv->dev, "PWON_IREF is 0x%x\n", val & BIT(7));
```

首先查看TXPLL_LOCK是否为1，如果不是，表明PHY没有lock完成。其次查看PWON_IREF是否为1，如果不为1，则表明PHY时钟异常。此时尝试切换combophy的时钟频率，修改rk3568.dtsi中的combphy2_psq的assigned-clock-rates，依次调整为25M或者100M进行尝试。

- 如果调整以上步骤均无效，请将PHY内部的时钟bypass到refclk差分信号脚上，进行测量。bypass加在rockchip_combphy_pcie_init函数的末尾，根据芯片的不同，设置如下代码所示

```
/* For RK356X, RK3588 */
u32 val;
val = readl(priv->mmio + (0xd << 2));
val |= BIT(5);
writel(val,priv->mmio + (0xd << 2));

/* For RK3528 */
u32 val;
val = readl(priv->mmio + 0x108);
val |= BIT(29);
writel(val, priv->mmio + 0x108);
```

设置完成后，请依次配置combphy2_psq的时钟频率为24M,25M以及100M，用示波器从PCIe的refclk差分信号脚上测量时钟情况，检查频率和幅值、抖动是否满足要求。

还需特别注意：由于PCIe 2.0接口与SATA或者USB接口复用，如果两者被错误地同时打开，开机过程或者休眠唤醒过程也会出现类似log：

```
naneng-combphy 2b060000.phy: phy type select 2 overwriting type 4
rk-pcie 2a210000.pcie: PHY is reused by other controller, check the dts
```

2.2 设备枚举和使用问题

2.2.1 PCIe外设Memory BAR资源分配异常

外设分配Memory资源异常主要分为三类：

- **地址空间超出平台限制**。各平台地址空间大小可查阅**常见应用问题**章节之"芯片支持分配的BAR空间地址域有多大"。此类异常的典型log如下所示，其特征关键字为`no space for`，表明21号总线外设向RK平台申请3GB的64bit memory空间，超出了限制导致无法分配资源。若为市售设备，将不受RK芯片支持；若为定制设备，请联系设备vendor确认是否可以修改其BAR空间容量编码。

```
3.286864] pci 0002:20:00.0: bridge configuration invalid ([bus 01-ff]),
reconfiguring
3.286886] scanning [bus 00-00] behind bridge, pass 1
3.288165] pci 0002:21 :00.0: supports D1 D2
3.288170] pci 0002:21 :00.0: PME# supported from DO D1 D3hot
3.298238] pci bus 0002:21: busn res: [bus 21-2f] end is updated to 21
3.298441] pci 0002:21:00.0: BAR 1: no space for [mem size 0xe0000000 ]
3.298456] pci 0002:21:00.0: BAR 1: failed to assign [mem size 0xe0000000 ]
3.298473] pci 0002:21:00.0: BAR 2: assigned [mem 0x380900000- 0x38090ffff pref ]
3.298488] pci 0002:21:00.0: PCI bridge to [bus 21]
```

- **class类型非法**。其特征关键字为`class 0x000000`，且lspci输出可见其BAR资源均显示unassigned状态。应对此问题的正确做法是联系模块供应商，对模块的class类型进行正确的配置。如果无法修改或者作为临时性的解决方案，可以参考所示补丁尝试对模块的class类型进行软件修复，需要填入此异常模块的VID和PID，并且class修复为其真实类型(请参考**常见应用问题**章节之"在系统中如何确定控制器与设备的对应关系"部分关于class类型的描述进行配置)。

```
[    2.335899] pci 0000:02:04.0: [13fe:1730] type 00 class 0x000000
[    2.335986] pci 0000:02:04.0: reg 0x10: [io  0x0000-0x00ff]
[    2.336023] pci 0000:02:04.0: reg 0x14: [mem 0x00000000-0x0000007f]
[    2.336058] pci 0000:02:04.0: reg 0x18: [mem 0x00000000-0x000000ff]
[    2.342340] pci_bus 0000:02: busn_res: [bus 02-ff] end is updated to 02
[    2.342444] pci 0000:00:00.0: BAR 8: assigned [mem 0xf4200000-0xf42fffff]
[    2.342484] pci 0000:00:00.0: BAR 6: assigned [mem 0xf4300000-0xf430ffff
pref]
[    2.342496] pci 0000:00:00.0: BAR 7: assigned [io  0x1000-0x1fff]
[    2.342510] pci 0000:01:00.0: BAR 8: assigned [mem 0xf4200000-0xf42fffff]
[    2.342519] pci 0000:01:00.0: BAR 7: assigned [io  0x1000-0x1fff]
[    2.342529] pci 0000:01:00.0: PCI bridge to [bus 02]
[    2.342546] pci 0000:01:00.0:   bridge window [io  0x1000-0x1fff]
[    2.342572] pci 0000:01:00.0:   bridge window [mem 0xf4200000-0xf42fffff]
[    2.342616] pci 0000:00:00.0: PCI bridge to [bus 01-ff]
[    2.342631] pci 0000:00:00.0:   bridge window [io  0x1000-0x1fff]
[    2.342645] pci 0000:00:00.0:   bridge window [mem 0xf4200000-0xf42fffff]
[    2.344896] pcieport 0000:00:00.0: Signaling PME with IRQ 87

lspci -vvv:
```

```
0000: 02:04.0 Non-VGA unclassified device: Advantech Co. Ltd Device 1730
Subsystem: Advantech Co. Ltd Device 1730
F1ags: medium devsel, IRQ 255
I/0 ports at[<unassigned> _disabled] [size=256]
Memory at <unassigned> (32-bit, non-prefetchabie) [disabled] [size=128]
Memory at <unassigned> (32-bit, non-prefetchable) [disabled] [size=256]
```

```
--- a/drivers/pci/quirks.c
+++ b/drivers/pci/quirks.c
@@ -207,6 +207,13 @@ static void quirk_mmio_always_on(struct pci_dev *dev)
 DECLARE_PCI_FIXUP_CLASS_EARLY(PCI_ANY_ID, PCI_ANY_ID,
                               PCI_CLASS_BRIDGE_HOST, 8, quirk_mmio_always_on);

+static void quirk_class_id_fixup(struct pci_dev *dev)
+{
+       dev->class = 0x123456; /* Fixup your own class id ! */
+}
+
+DECLARE_PCI_FIXUP_CLASS_EARLY(PCI_VENDOR_ID_FOO, PCI_DEVICE_ID_BAR, 8,
quirk_class_id_fixup); /* Please Add Correct Vendor ID and Device ID ! */
```

- **外设的BAR寄存器校验异常**。其特征关键字为`assigned [mem`和`error updating`，且lspci输出可见其BAR资源均显示unassigned状态。出现此问题的原因是外设工作状态异常，导致BAR地址写入外设后，回读校验失败。其可能的因素是低功耗支持异常，或可能是外设对于非Gen1的RC支持存在问题，亦或者是其他类型的模块工作异常。

```
[    2.460092] pci 0002:21:00.0: calc_l1ss_pwron: Invalid T_PwrOn scale: 3
[    2.472049] pci_bus 0002:21: busn_res: [bus 21-2f] end is updated to 21
[    2.472089] pci 0002:20:00.0: BAR 8: assigned [mem 0xf2200000-0xf23fffff]
[    2.472105] pci 0002:20:00.0: BAR 6: assigned [mem 0xf2400000-0xf240ffff
pref]
[    2.472124] pci 0002:21:00.0: BAR 0: assigned [mem 0xf2200000-0xf23fffff
64bit]
[    2.472139] pci 0002:21:00.0: BAR 0: error updating (0xf2200004 !=
0xffffffff)
[    2.472153] pci 0002:21:00.0: BAR 0: error updating (high 0x000000 !=
0xffffffff)
```

此类问题，首先请测量模组供电、PERSTN复位信号和100M参考时钟三者的控制时序是否满足模组手册要求，例如QCNFA765模块如果使用常供电方式，就使得时序异常进而导致模块工作异常，产生此问题。如果硬件外围设计与接口时序均满足模块手册要求，则建议依次尝试下列的补丁。若依然无效，联系供应商协助分析模块异常原因。

1. 禁止此模块的ASPM功能，其中PID和VID需要填写设备真实的ID。

```
--- a/drivers/pci/quirks.c
+++ b/drivers/pci/quirks.c
@ -2356,6 +2356,7 @ static void quirk_disable_aspm_l0s_l1(struct pci_dev *dev) *
disable both L0s and L1 for now to be safe.
*/
DECLARE_PCI_FIXUP_FINAL(PCI_VENDOR_ID_ASMEDIA, 0x1080,
quirk_disable_aspm_l0s_l1);
+DECLARE_PCI_FIXUP_FINAL(VID, PID, quirk_disable_aspm_l0s_l1);  /* Please Add
Correct Vendor ID and Device ID to disable L0s and L1 ASPM */
```

2. 限制对应控制的模式为Gen1，在dtsi中修改对应节点的max-link-speed，示例如下：

```
--- a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
@@ -4552,7 +4552,7 @@
        num-ib-windows = <8>;
        num-ob-windows = <8>;
        num-viewport = <4>;
-       max-link-speed = <2>;
+       max-link-speed = <1>;
        msi-map = <0x4000 &its0 0x4000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy0_ps PHY_TYPE_PCIE>;
```

3. 增加外设供电稳定时间以及PERSTN复位时间，示例如下：

```
--- a/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
@@ -159,7 +159,7 @@
                regulator-max-microvolt = <3300000>;
                enable-active-high;
                gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
-               startup-delay-us = <5000>;
+               startup-delay-us = <500000>;
                vin-supply = <&vcc12v_dcin>;
        }; /* vcc3v3_pcie30 */

@@ -551,6 +551,7 @@
 &pcie3x4 {
        reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
        vpcie3v3-supply = <&vcc3v3_pcie30>;
+       rockchip,perst-inactive-ms = <1000>;
        status = "okay";
 };
```

4. 已知高通部分系列的wifi模块存在异常，受影响的模块包AR9xxx系列和QCA9xxx系列。非高通系列模块无需此补丁。

```
--- a/drivers/pci/pcie/aspm.c
+++ b/drivers/pci/pcie/aspm.c
@@ -192,12 +192,56 @@ static void pcie_clkpm_cap_init(struct pcie_link_state
*link, int blacklist)
    link->clkpm_disable = blacklist ? 1 : 0;
 }

+static int pcie_downgrade_link_to_gen1(struct pci_dev *parent)
+{
+   u16 reg16;
+   u32 reg32;
+   int ret;
+
+   /* Check if link is capable of higher speed than 2.5 GT/s */
+   pcie_capability_read_dword(parent, PCI_EXP_LNKCAP, &reg32);
+   if ((reg32 & PCI_EXP_LNKCAP_SLS) <= PCI_EXP_LNKCAP_SLS_2_5GB)
+       return 0;
+
```

```
+	/* Check if link speed can be downgraded to 2.5 GT/s */
+	pcie_capability_read_dword(parent, PCI_EXP_LNKCAP2, &reg32);
+	if (!(reg32 & PCI_EXP_LNKCAP2_SLS_2_5GB)) {
+		pci_err(parent, "ASPM: Bridge does not support changing Link Speed to
2.5 GT/s\n");
+		return -EOPNOTSUPP;
+	}
+
+	/* Force link speed to 2.5 GT/s */
+	ret = pcie_capability_clear_and_set_word(parent, PCI_EXP_LNKCTL2,
+						 PCI_EXP_LNKCTL2_TLS,
+						 PCI_EXP_LNKCTL2_TLS_2_5GT);
+	if (!ret) {
+		/* Verify that new value was really set */
+		pcie_capability_read_word(parent, PCI_EXP_LNKCTL2, &reg16);
+		if ((reg16 & PCI_EXP_LNKCTL2_TLS) != PCI_EXP_LNKCTL2_TLS_2_5GT)
+			ret = -EINVAL;
+	}
+
+	if (ret) {
+		pci_err(parent, "ASPM: Changing Target Link Speed to 2.5 GT/s failed:
%d\n", ret);
+		return ret;
+	}
+
+	pci_info(parent, "ASPM: Target Link Speed changed to 2.5 GT/s due to
quirk\n");
+	return 0;
+}
+
 static bool pcie_retrain_link(struct pcie_link_state *link)
 {
     struct pci_dev *parent = link->pdev;
     unsigned long end_jiffies;
     u16 reg16;

+	if ((link->downstream->dev_flags &
PCI_DEV_FLAGS_NO_RETRAIN_LINK_WHEN_NOT_GEN1) &&
+	    pcie_downgrade_link_to_gen1(parent)) {
+		pci_err(parent, "ASPM: Retrain Link at higher speed is disallowed by
quirk\n");
+		return false;
+	}
+
     pcie_capability_read_word(parent, PCI_EXP_LNKCTL, &reg16);
     reg16 |= PCI_EXP_LNKCTL_RL;
     pcie_capability_write_word(parent, PCI_EXP_LNKCTL, reg16);
diff --git a/drivers/pci/quirks.c b/drivers/pci/quirks.c
index 653660e3ba9e..4999ad9d08b8 100644
--- a/drivers/pci/quirks.c
+++ b/drivers/pci/quirks.c
@@ -3553,23 +3553,46 @@ static void mellanox_check_broken_intx_masking(struct
pci_dev *pdev)
 DECLARE_PCI_FIXUP_FINAL(PCI_VENDOR_ID_MELLANOX, PCI_ANY_ID,
			mellanox_check_broken_intx_masking);

-static void quirk_no_bus_reset(struct pci_dev *dev)
+static void quirk_no_bus_reset_and_no_retrain_link(struct pci_dev *dev)
```

```
 {
-	dev->dev_flags |= PCI_DEV_FLAGS_NO_BUS_RESET;
+	dev->dev_flags |= PCI_DEV_FLAGS_NO_BUS_RESET |
+			  PCI_DEV_FLAGS_NO_RETRAIN_LINK_WHEN_NOT_GEN1;
 }

 /*
- * Some Atheros AR9xxx and QCA988x chips do not behave after a bus reset.
+ * Atheros AR9xxx and QCA9xxx chips do not behave after a bus reset and also
+ * after retrain link when PCIe bridge is not in GEN1 mode at 2.5 GT/s speed.
  * The device will throw a Link Down error on AER-capable systems and
  * regardless of AER, config space of the device is never accessible again
  * and typically causes the system to hang or reset when access is attempted.
+ * Or if config space is accessible again then it contains only dummy values
+ * like fixed PCI device ID 0xABCD or values not initialized at all.
+ * Retrain link can be called only when using GEN1 PCIe bridge or when
+ * PCIe bridge has forced link speed to 2.5 GT/s via PCI_EXP_LNKCTL2 register.
+ * To reset these cards it is required to do PCIe Warm Reset via PERST# pin.
  * https://lore.kernel.org/r/20140923210318.498dacbd@dualc.maya.org/
+ * https://lore.kernel.org/r/87h7l8axqp.fsf@toke.dk/
+ * https://www.mail-archive.com/ath9k-devel@lists.ath9k.org/msg07529.html
  */
-DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0030, quirk_no_bus_reset);
-DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0032, quirk_no_bus_reset);
-DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x003c, quirk_no_bus_reset);
-DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0033, quirk_no_bus_reset);
-DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0034, quirk_no_bus_reset);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x002e,
+			 quirk_no_bus_reset_and_no_retrain_link);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0030,
+			 quirk_no_bus_reset_and_no_retrain_link);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0032,
+			 quirk_no_bus_reset_and_no_retrain_link);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0033,
+			 quirk_no_bus_reset_and_no_retrain_link);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0034,
+			 quirk_no_bus_reset_and_no_retrain_link);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x003c,
+			 quirk_no_bus_reset_and_no_retrain_link);
+DECLARE_PCI_FIXUP_HEADER(PCI_VENDOR_ID_ATHEROS, 0x0042,
+			 quirk_no_bus_reset_and_no_retrain_link);
+
+static void quirk_no_bus_reset(struct pci_dev *dev)
+{
+	dev->dev_flags |= PCI_DEV_FLAGS_NO_BUS_RESET;
+}

 /*
  * Root port on some Cavium CN8xxx chips do not successfully complete a bus
diff --git a/include/linux/pci.h b/include/linux/pci.h
index 86c799c97b77..fdbf7254e4ab 100644
--- a/include/linux/pci.h
+++ b/include/linux/pci.h
@@ -227,6 +227,8 @@ enum pci_dev_flags {
     PCI_DEV_FLAGS_NO_FLR_RESET = (__force pci_dev_flags_t) (1 << 10),
     /* Don't use Relaxed Ordering for TLPs directed at this device */
     PCI_DEV_FLAGS_NO_RELAXED_ORDERING = (__force pci_dev_flags_t) (1 << 11),
     /* Device does honor MSI masking despite saying otherwise */
```

```
    PCI_DEV_FLAGS_HAS_MSI_MASKING = (__force pci_dev_flags_t) (1 << 12),
+   /* Don't Retrain Link for device when bridge is not in GEN1 mode */
+   PCI_DEV_FLAGS_NO_RETRAIN_LINK_WHEN_NOT_GEN1 = (__force pci_dev_flags_t) (1
<< 13),
 };
```

- **switch的无效端口占用了部分资源**。以RK3588为例，其每个root port可分配的32-bit BAR空间仅为16MB。有的switch，如ASM2812，不论port的下游是否有设备，都会如下面log所示，申请2MB的32-bit BAR空间。根据lspci结果我们可以看到，bus 12的switch下的2/3/a/b这四个port下面并无设备，但却额外占用了总共8MB的32-bit BAR空间，导致正常设备的32-bit BAR资源不够分配。

```
[   22.005666] pci 0001:12:00.0: BAR 8: no space for [mem size 0x00600000]
[   22.040400] pci 0001:12:00.0: BAR 8: failed to assign [mem size 0x00600000]
[   22.040402] pci 0001:12:08.0: BAR 8: no space for [mem size 0x00600000]
[   22.040406] pci 0001:12:08.0: BAR 8: failed to assign [mem size 0x00600000]
[   22.067370] pci 0001:12:00.0: BAR 9: assigned [mem 0x940000000-0x9401fffff
64bit pref]
[   22.067373] pci 0001:12:02.0: BAR 8: no space for [mem size 0x00200000]
[   22.080018] pci 0001:12:02.0: BAR 8: failed to assign [mem size 0x00200000]
[   22.080022] pci 0001:12:02.0: BAR 9: assigned [mem 0x940200000-0x9403fffff
64bit pref]
[   22.093658] pci 0001:12:03.0: BAR 8: no space for [mem size 0x00200000]
[   22.093661] pci 0001:12:03.0: BAR 8: failed to assign [mem size 0x00200000]
[   22.093663] pci 0001:12:03.0: BAR 9: assigned [mem 0x940400000-0x9405fffff
64bit pref]
[   22.106310] pci 0001:12:08.0: BAR 9: assigned [mem 0x940600000-0x9407fffff
64bit pref]
[   22.106313] pci 0001:12:0a.0: BAR 8: no space for [mem size 0x00200000]
[   22.120756] pci 0001:12:0a.0: BAR 8: failed to assign [mem size 0x00200000]
[   22.130702] pci 0001:12:0a.0: BAR 9: assigned [mem 0x940800000-0x9409fffff
64bit pref]
[   22.130705] pci 0001:12:0b.0: BAR 8: no space for [mem size 0x00200000]
[   22.144573] pci 0001:12:0b.0: BAR 8: failed to assign [mem size 0x00200000]

lspci -vvt
-+-[0003:30]---00.0-[31]----00.0  Realtek Semiconductor Co., Ltd. Device
[10ec:8125]
 +-[0001:10]---00.0-[11-18]----00.0-[12-18]--+-00.0-[13]----00.0  Aquantia Corp.
Device
 |                                          +-02.0-[14]--
 |                                          +-03.0-[15]--
 |                                          +-08.0-[16]----00.0  Aquantia Corp.
Device
 |                                          +-0a.0-[17]--
 |                                          \-0b.0-[18]--
 \-[0000:00]---00.0-[01-ff]----00.0  ASMedia Technology Inc. Device [1b21:1164]

lspci -nn
0000:00:00.0 PCI bridge [0604]: Fuzhou Rockchip Electronics Co., Ltd Device
[1d87:3588] (rev 01)
0000:01:00.0 SATA controller [0106]: ASMedia Technology Inc. Device [1b21:1164]
(rev 02)
0001:10:00.0 PCI bridge [0604]: Fuzhou Rockchip Electronics Co., Ltd Device
[1d87:3588] (rev 01)
0001:11:00.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
```

```
0001:12:00.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
0001:12:02.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
0001:12:03.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
0001:12:08.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
0001:12:0a.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
0001:12:0b.0 PCI bridge [0604]: ASMedia Technology Inc. Device [1b21:2812] (rev
01)
0001:13:00.0 Ethernet controller [0200]: Aquantia Corp. Device [1d6a:14c0] (rev
03)
0001:16:00.0 Ethernet controller [0200]: Aquantia Corp. Device [1d6a:14c0] (rev
03)
0003:30:00.0 PCI bridge [0604]: Fuzhou Rockchip Electronics Co., Ltd Device
[1d87:3588] (rev 01)
0003:31:00.0 Ethernet controller [0200]: Realtek Semiconductor Co., Ltd. Device
[10ec:8125] (rev 05)
```

处理这类问题，建议同switch厂家协商，是否可以修改switch的固件配置，关闭不存在下游设备的端口的BAR空间。如果switch的固件无法关闭的情况下，且产品的拓扑结构相对固定（即不存在switch的各插槽随机插入设备的情况），可以尝试下列补丁进行过滤：

```
diff --git a/drivers/pci/probe.c b/drivers/pci/probe.c
index ece90a2..53bef6b 100644
--- a/drivers/pci/probe.c
+++ b/drivers/pci/probe.c
@@ -2794,6 +2794,37 @@ void __weak pcibios_fixup_bus(struct pci_bus *bus)
        /* nothing to do, expected to be removed in the future */
 }

+struct scan_blacklist_devfn {
+       u8 bus; /* bus number */
+       u8 dev; /* device number */
+       u8 func; /* function number */
+};
+
+#define ANY 0xff
+
+/* 请在sbl表中填写实际要过滤的设备bdf，以上述log为例，过滤的是bus12的2/3/a/b四个端口*/
+static struct scan_blacklist_devfn sbl[] = {
+       {12, 0x2, ANY},
+       {12, 0x3, ANY},
+       {12, 0xa, ANY},
+       {12, 0xb, ANY},
+};
+
+static bool pci_scan_check_blacklist(u8 bus, unsigned int devfn)
+{
+       int i;
+
+       for (i = 0; i < ARRAY_SIZE(sbl); i++) {
+               if (bus == sbl[i].bus && PCI_SLOT(devfn) == sbl[i].dev) {
+                       if (sbl[i].func == ANY)
+                               return true;
```

```
+                       else
+                               return sbl[i].func == PCI_FUNC(devfn);
+               }
+       }
+
+       return false;
+}
+
 /**
  * pci_scan_child_bus_extend() - Scan devices below a bus
  * @bus: Bus to scan for devices
@@ -2819,6 +2850,9 @@ static unsigned int pci_scan_child_bus_extend(struct
pci_bus *bus,

        /* Go find them, Rover! */
        for (devfn = 0; devfn < 256; devfn += 8) {
+               if (pci_scan_check_blacklist(bus->number, devfn))
+                       continue;
+
                nr_devs = pci_scan_slot(bus, devfn);
```

2.2.2 PCIe外设IO BAR资源分配异常

在枚举过程中出现以下IO BAR地址空间写入异常，并报有类似错误log，其特征关键字为`assigned [io`和`error updating`，或者`firmware Bug`，则建议核对设备手册中关于IO BAR地址空间的限制：

```
dmesg | grep "0002:22"
[    2.324600] pci_bus 0002:22: extended config space not accessible
[    2.324764] pci 0002:22:00.0: [13f6:0111] type 00 class 0x040100
[    2.324873] pci 0002:22:00.0: [Firmware Bug]: reg 0x10: invalid BAR (can't
size)
[    2.325445] pci 0002:22:00.0: supports D1 D2
[    2.331041] pci_bus 0002:22: busn_res: [bus 22-2f] end is updated to 22
```

或：

```
console:/ $ dmesg | grep "0002:22"
[    2.434085] [  T117] pci_bus 0002:22: extended config space not accessible
[    2.434629] [  T117] pci 0002:22:00.0: [13f6:0111] type 00 class 0x040100
[    2.434842] [  T117] pci 0002:22:00.0: reg 0x10: initial BAR value 0x0000d000
invalid
[    2.434869] [  T117] pci 0002:22:00.0: reg 0x10: [io  size 0x0100]
[    2.435536] [  T117] pci 0002:22:00.0: supports D1 D2
[    2.458229] [  T117] pci_bus 0002:22: busn_res: [bus 22-2f] end is updated to
22
[    2.458542] [  T117] pci 0002:22:00.0: BAR 0: assigned [io  0x1000-0x10ff]
[    2.458581] [  T117] pci 0002:22:00.0: BAR 0: error updating (0x80801001 !=
0x001001)
[    2.463002] [  T117] snd_cmipci 0002:22:00.0: enabling device (0080 -> 0081)
```

注释：

- 以上log为使用PCI声卡CMI8738时的异常打印。PCI原生设计是针对X86体系，所以IO BAR地址长度通常较小。CMI8738声卡IO BAR0地址限制在 0xFF00 以内，所以可参考以下补丁：

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3568.dtsi
b/arch/arm64/boot/dts/rockchip/rk3568.dtsi
index 9dc057637177..a060dcbf7df5 100644
--- a/arch/arm64/boot/dts/rockchip/rk3568.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568.dtsi
@@ -2311,7 +2311,7 @@
                phy-names = "pcie-phy";
                power-domains = <&power RK3568_PD_PIPE>;
                ranges = <0x00000800 0x0 0x80000000 0x3 0x80000000 0x0 0x800000
-                         0x81000000 0x0 0x80800000 0x3 0x80800000 0x0 0x100000
+                         0x81000000 0x0 0x00000000 0x3 0x80800000 0x0 0x100000
                          0xc3000000 0x0 0x80900000 0x3 0x80900000 0x0
0x3f700000>;
                reg = <0x3 0xc0800000 0x0 0x400000>,
                      <0x0 0xfe280000 0x0 0x10000>;
```

2.2.3 无法读取配给外设的IO地址空间

```
0002:21:00.0 Serial controller: Device 1c00:3853 (rev 10) (prog-if 05 [16850])
        Subsystem: Device 1c00:3853
        Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr-
Stepping- SERR- FastB2B- DisINTx-
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort-
<MAbort- >SERR- <PERR- INTx-
        Interrupt: pin A routed to IRQ 0
        Region 0: I/O ports at 1000 [disabled] [size=256]
        Region 1: Memory at 380900000 (32-bit, prefetchable) [disabled]
[size=32K]
        Region 2: I/O ports at 100100 [disabled] [size=4]
        [virtual] Expansion ROM at 380908000 [disabled] [size=32K]
        Capabilities: [60] Power Management version 3

0002:21:00.0 Serial Controller: device 1c00:3853 (rev 10)
00: 00 1c 53 38 00 00 10 00 10 05 00 07 00 00 00 00
10: 01 00 80 80 08 00 90 80 01 01 80 80 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 1c 53 38
30: 00 00 00 00 60 00 00 00 00 00 00 00 00 00 01 00 00

console:/ # cat /proc/ioports
00000000-000fffff : I/O
  00001000-00001fff : PCI Bus 0002:21
    00001000-00001007 : 0002:21:00.0
    00001008-0000100f : 0002:21:00.2
    00001010-00001017 : 0002:21:00.2
```

可以看到，lspci显示出此外设的IO端口分配地址是0x1000，而对应的pcie bus为0x80800000。这与rk3568.dtsi中定义的不符。以pcie3x2为例，其定义的IO端口的物理起始地址是0x380800000，所对应的pci bus起始地址为0x80800000。因此不能直接用IO命令或者devmem命令访问0x1000这个物理地址，或者软件访问其ioremap后的地址，否则会发生异常。

出现这种情况是由于历史原因，x86的PCIe中IO端口地址与内存端口地址是分离的，16M以下的地址段为IO端口地址。而ARM平台使用的PCIe IO地址端口是由内存端口模拟而来的。因此PCIe协议栈在计算IO端口地址的时候，为了将其地址也限制在16M以下，形式上看起来与x86平台更一致化，故而做了一层转换。转换的原理是将4K对齐处的0x1000作为IO端口的起始地址。换句话说，lspci中看到的IO端口地址0x1000即对应0x380800000这个物理地址。所以如果用IO命令或者devmem命令，可直接访问0x380800000。以此类推，如果分配到的IO端口地址为0x1010, 则根据该原理可得出所需访问的真实物理地址为0x380800010。如果是自行编写的驱动中，想要获取真实的IO端口地址进行访问，可以参考8250_port.c与8250_pci.c 串口驱动，使用pci_ioremap_bar和request_region函数组合，获取IO端口的真实物理地址以及iomap后的CPU地址。

2.2.4 PCIe 设备可以Link却无法枚举出设备

若出现设备可以Link up但是却没有枚举出设备，例如lspci可见root port所在bus 0却看不到下一级bus 1等。

```
rk-pcie fe150000.pcie: PCIe Link up, LTSSM is 0x30011
rk-pcie fe150000.pcie: PCI host bridge to bus 0000:00
pci_bus 0000:00: root bus resource [bus 00-0f]
pci_bus 0000:00: root bus resource [??? 0xf0000000-0xf00fffff flags 0x0]
pci_bus 0000:00: root bus resource [io  0x300000-0x3fffff] (bus address
[0xf0100000-0xf01fffff])
pci_bus 0000:00: root bus resource [mem 0xf0200000-0xf0ffffff]
pci_bus 0000:00: root bus resource [mem 0x900000000-0x93fffffff pref]
pci 0000:00:00.0: [1d87:3588] type 01 class 0x060400
pci 0000:00:00.0: reg 0x10: [mem 0x00000000-0x3fffffff]
pci 0000:00:00.0: reg 0x14: [mem 0x00000000-0x3fffffff]
pci 0000:00:00.0: reg 0x38: [mem 0x00000000-0x0000ffff pref]
pci 0000:00:00.0: supports D1 D2
pci 0000:00:00.0: PME# supported from D0 D1 D3hot

lspci
0000:00:00.0 PCI bridge: Fuzhou Rockchip Electronics Co., Ltd Device 3588 (rev
01) (prog-if 00 [Normal decode])
```

此原因是读取的下一级设备，即bus 1设备的vendor ID 为非法数值。请在drivers/pci/probe.c文件的pci_bus_generic_read_dev_vendor_id函数中添加打印确认，协议栈默认了四种非法ID类型。

```
--- a/drivers/pci/probe.c
+++ b/drivers/pci/probe.c
@@ -2314,6 +2314,8 @@ bool pci_bus_generic_read_dev_vendor_id(struct pci_bus
*bus, int devfn, u32 *l,
        if (pci_bus_read_config_dword(bus, devfn, PCI_VENDOR_ID, l))
                return false;

+       dev_info(&bus->dev, "vendor id is 0x%x\n", *l);
+
        /* Some broken boards return 0 or ~0 if a slot is empty: */
        if (*l == 0xffffffff || *l == 0x00000000 ||
            *l == 0x0000ffff || *l == 0xffff0000)
```

目前已知ZX-200这款switch芯片在释放复位后需要加载bin并运行的时间较长，可能出现link up之后还未完全运行好其内部程序，导致RC去枚举时候读取到0的非法ID。如有类似情况请加上下列补丁尝试修复，如补丁加入仍无法解决问题，则建议联系设备厂家协助排查具体原因。

```
--- a/drivers/pci/controller/dwc/pcie-dw-rockchip.c
+++ b/drivers/pci/controller/dwc/pcie-dw-rockchip.c
@@ -814,7 +814,7 @@ static int rk_pcie_establish_link(struct dw_pcie *pci)
                         * that LTSSM max timeout is 24ms per period, we can
wait a bit
                         * more for Gen switch.
                         */
-                       msleep(50);
+                       msleep(1000);
                        /* In case link drop after linkup, double check it */
                        if (dw_pcie_link_up(pci)) {
                                dev_info(pci->dev, "PCIe Link up, LTSSM is
0x%x\n",
```

2.2.5 PCIe 设备可以枚举但访问异常

部分设备PERSTN复位时间不够，可能概率性导致其虽然可以被系统枚举，但是实际工作不正常。例如有客户使用PCIe switch扩展后再连RTL8111网卡，出现如下的异常信息。此时需要增加PERSTN的复位时间，在DTS中增加rockchip,perst-inactive-ms属性。详情请参照DTS property说明章节。

```
[    8.739794] enP4p67s0f0: 0xffffffc0127bd000, 86:41:ff:d2:48:a0, IRQ 133
[   10.178084] ------------[ cut here ]------------
[   10.178100] WARNING: CPU: 6 PID: 691 at
drivers/net/ethernet/realtek/r8168/r8168_n.c:7291
rtl8168_wait_phy_ups_resume+0x6c/0x88
[   10.178104] Modules linked in:
[   10.178112] CPU: 6 PID: 691 Comm: dhcpcd Not tainted 5.10.66 #1
[   10.178116] Hardware name: Rockchip RK3588 NVR DEMO LP4 V10 Board (DT)
[   10.178121] pstate: 60400089 (nZCv daIf +PAN -UAO -TCO BTYPE=--)
[   10.178127] pc : rtl8168_wait_phy_ups_resume+0x6c/0x88
[   10.178132] lr : rtl8168_wait_phy_ups_resume+0x54/0x88
[   10.178135] sp : ffffffc01358ba30
[   10.178139] x29: ffffffc01358ba30 x28: 0000000000001043
[   10.178145] x27: 0000000000000000 x26: 0000000000008914
[   10.178151] x25: 0000000000000000 x24: ffffff8102e10c38
[   10.178157] x23: 0000000000418958 x22: 0000000000000002
[   10.178163] x21: ffffff8102e109c0 x20: 0000000000000064
[   10.178168] x19: 0000000000000007 x18: 0000000000000000
[   10.178174] x17: 0000000000000000 x16: 0000000000000000
[   10.178179] x15: 000000000000000a x14: 00000000000b49d2
[   10.178186] x13: 0000000000000006 x12: ffffffffffffffff
[   10.178191] x11: 0000000000000010 x10: ffffffc09358b767
[   10.178197] x9 : ffffffc0104d3868 x8 : 0000000000000000
[   10.178202] x7 : 663a31343a363820 x6 : 00000000ffff0000
[   10.178208] x5 : 0000000000000004 x4 : 000000000000ffff
[   10.178213] x3 : 0000000000000006 x2 : ffffffc011dcbbb8
[   10.178219] x1 : ffffffc01358b9f0 x0 : 0000000000005dc3
[   10.178225] Call trace:
[   10.178231]  rtl8168_wait_phy_ups_resume+0x6c/0x88
[   10.178236]  rtl8168_exit_oob+0x2e8/0x36c
[   10.178241]  rtl8168_open+0x1c8/0x37c
[   10.178247]  __dev_open+0x154/0x17c
[   10.178252]  __dev_change_flags+0xfc/0x1ac
[   10.178257]  dev_change_flags+0x30/0x70
[   10.178263]  devinet_ioctl+0x288/0x51c
```

```
[   10.178268]  inet_ioctl+0x1b8/0x1e8

lspci -vvv的输出结果：
0003:31:00.0 Class 0108: Device 8086:f1a5 (rev ff) (prog-if ff)
        !!! Unknown header type 7f
```

2.2.6 外设枚举后通信过程中报错

以下是NVMe在RK3566-EVB2上进行正常枚举之后，通信过程中突然设备异常报错的log。不论是什么设备，如果可以正常枚举并使能，则可以看到类似nvme 0000:01:00.0: enabling device (0000 -> 0002)的log。此后通信过程中设备报错，需要考虑如下三个方面：

- 利用示波器测量触发外设的电源，排除是否有跌落的情况发生
- 利用示波器测量触发外设的PERSTN信号，排除是否被人误操作导致设备被复位的情况发生
- 利用示波器测量触发PCIe PHY的0v9和1v8两路电源，排除是否PHY的电源异常

特别提醒：RK EVB有较多的信号复用，利用拨码将PCIe的PERSTN控制信号和其他外设的IO进行复用，请配合硬件重点确认。例如目前已知有部分RK3566-EVB2的拨码有异常，需要修正。

```
[    2.426038] pci 0000:00:00.0:   bridge window [mem 0x300900000-0x3009fffff]
[    2.426183] pcieport 0000:00:00.0: of_irq_parse_pci: failed with rc=-22
[    2.427493] pcieport 0000:00:00.0: Signaling PME with IRQ 106
[    2.427712] pcieport 0000:00:00.0: AER enabled with IRQ 115
[    2.427899] pcieport 0000:00:00.0: of_irq_parse_pci: failed with rc=-22
[    2.428202] nvme nvme0: pci function 0000:01:00.0
[    2.428259] nvme 0000:01:00.0: enabling device (0000 -> 0002)
[    2.535404] nvme nvme0: missing or invalid SUBNQN field.
[    2.535522] nvme nvme0: Shutdown timeout set to 8 seconds
...
[   48.129408] print_req_error: I/O error, dev nvme0n1, sector 0
[   48.137197] nvme 0000:01:00.0: enabling device (0000 -> 0002)
[   48.137299] nvme nvme0: Removing after probe failure status: -19
[   48.147182] Buffer I/O error on dev nvme0n1, logical block 0, async page read
[   48.162900] nvme nvme0: failed to set APST feature (-19)
```

2.2.7 外设枚举过程报FW异常

如设备在枚举过程分配BAR空间报如下两类错误，一般问题是设备的BAR空间与协议不兼容，需要特殊处理。需要修改drivers/pci/quirks.c中，增加对应quirk处理。具体信息应该咨询设备厂商。目前已知JMB585芯片给出的解决办法是需要重复读取BAR空间，才可以解决他们Fireware的异常，那么可以使用echo 1 > /sys/bus/pci/rescan重新对链路进行扫描，可以修复。

```
类型1：
[    2.379768] rk-pcie 3c0000000.pcie: PCIe Link up, LTSSM is 0x30011
[    2.380155] rk-pcie 3c0000000.pcie: PCI host bridge to bus 0000:00
[    2.380187] pci_bus 0000:00: root bus resource [bus 00-0f]
[    2.380204] pci_bus 0000:00: root bus resource [??? 0x300000000-0x3007fffff
flags 0x0] (bus address [0x00000000-0x007fffff])
[    2.380217] pci_bus 0000:00: root bus resource [io  0x0000-0xfffff] (bus
address [0x800000-0x8fffff])
[    2.380230] pci_bus 0000:00: root bus resource [mem 0x300900000-0x33fffffff]
(bus address [0x00900000-0x3fffffff])
```

```
[    2.394983] pci 0000:01:00.0: [Firmware Bug] reg 0x10: invalid BAR (can't
size)
```

类型2：

```
[    2.548219] pci 0000:01:00.0: [10ec:b723] type 00 class 0x028000
[    2.548389] pci 0000:01:00.0: reg 0x10: initial BAR value 0x00000000 invalid
[    2.548426] pci 0000:01:00.0: reg 0x10: [io  size 0x0100]
[    2.548549] pci 0000:01:00.0: reg 0x18: [mem 0x00000000-0x00003fff 64bit]
[    2.549132] pci 0000:01:00.0: supports D1 D2
[    2.549138] pci 0000:01:00.0: PME# supported from D0 D1 D2 D3hot D3cold
```

2.2.8 重新映射后访问PCIe设备的BAR地址空间异常

1. 首先，如果内核中利用ioremap将分配给PCIe外设的BAR地址进行映射后，使用memset或者memcpy来读写，会产生alignment fault错误。亦或者利用mmap将分配给PCIe外设的BAR地址映射到用户态进行访问，使用memset或者memcpy来读写，会产生sigbug错误。原因是memcpy有对齐优化问题或者memset在ARM64上会使用类似DC ZVA等指令，这些指令不支持Device memory type(nGnRE)。

```
[    69.195811] Unhandled fault: alignment fault (0x96000061) at
0xffffff8009800000
[    69.195829] Internal error: : 96000061 [#1] PREEMPT SMP
[    69.363352] Modules linked in:
[    69.363655] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 4.19.172 #691
[    69.364205] Hardware name: Rockchip rk3568 evb board (DT)
[    69.364688] task: ffffffc00a300000 task.stack: ffffffc00a2dc000
[    69.365227] PC is at __memset+0x16c/0x190
[    69.365593] LR is at snda_alloc_res+0xac/0xfc
[    69.366054] pc : [<ffffff800839a2ac>] lr : [<ffffff80085055b8>] pstate:
404000c5
[    69.366713] sp : ffffffc00a2df810
```

解决办法：改用memset_io或者memset_fromio/memset_toio等API。如果需要在用户态操作，请使用循环赋值方式来拷贝或者清理内存。另外，由于对齐访问的要求，如果您的程序触发了上述异常，强烈建议在内核配置中开启这个配置CONFIG_ROCKCHIP_ARM64_ALIGN_FAULT_FIX。

2. 其次，针对RK3588如果对BAR空间赋值操作触发以下类似异常，这是由于编译器将四个地址连续的访问优化成一个平台不支持的16字节的访问。

```
[14037.477507][    C3] SError Interrupt on CPU3, code 0xbe000011 -- SError
[14037.477512][    C3] CPU: 3 PID: 2037 Comm: memtest Not tainted 5.10.110 #1690
[14037.477514][    C3] Hardware name: Rockchip RK3588 EVB1 LP4 V10 Board (DT)
[14037.477516][    C3] pstate: 00001000 (nzcv daif -PAN -UAO -TCO BTYPE=--)
[14037.477518][    C3] pc : 0000000000405d58
[14037.477519][    C3] lr : 0000000000405fb4
[14037.477521][    C3] sp : 0000007fc48effd0
[14037.477522][    C3] x29: 0000007fc48effd0 x28: 0000000000000000
[14037.477529][    C3] x27: 0000000000452000 x26: 000000000048b000
[14037.477533][    C3] x25: 000000000048b000 x24: 0000000000000018
[14037.477537][    C3] x23: 0000000000489030 x22: 0000000000400280
[14037.477541][    C3] x21: 0000000000000000 x20: 0000000000400eb8
[14037.477545][    C3] x19: 0000000000400df0 x18: 0000000000000000
[14037.477549][    C3] x17: 0000000000417e00 x16: 0000000000489030
[14037.477554][    C3] x15: 000000000572c738 x14: 0000000000000000
[14037.477558][    C3] x13: 0000000000000150 x12: 000000000048b000
```

```
[14037.477562][    C3] x11: 0000000000000000 x10: 0000200000000000
[14037.477566][    C3] x9 : 00003fffffffffff x8 : 00000000000000de
[14037.477570][    C3] x7 : 0000000000000000 x6 : 000000000048af30
[14037.477574][    C3] x5 : 00000000f0000000 x4 : 0000000000000000
[14037.477578][    C3] x3 : 0000000000000001 x2 : 0000000000000001
[14037.477582][    C3] x1 : 0000000000489058 x0 : 0000000000000000
[14037.477587][    C3] Kernel panic - not syncing: Asynchronous SError Interrupt
[14037.477589][    C3] CPU: 3 PID: 2037 Comm: memtest Not tainted 5.10.110 #1690
[14037.477590][    C3] Hardware name: Rockchip RK3588 EVB1 LP4 V10 Board (DT)
[14037.477592][    C3] Call trace:
[14037.477593][    C3]  dump_backtrace+0x0/0x1a8
[14037.477594][    C3]  show_stack+0x18/0x28
[14037.477596][    C3]  dump_stack_lvl+0xcc/0xf4
[14037.477598][    C3]  dump_stack+0x18/0x58
[14037.477600][    C3]  panic+0x170/0x36c
[14037.477602][    C3]  nmi_panic+0x8c/0x90
[14037.477603][    C3]  arm64_serror_panic+0x78/0x84
[14037.477605][    C3]  arm64_is_fatal_ras_serror+0x34/0xc8
[14037.477607][    C3]  do_serror+0x6c/0x98
[14037.477608][    C3]  el0_error_naked+0x14/0x1c
[14037.477622][    C2] CPU2: stopping
[14037.477684][    C0] CPU0: stopping
[14037.477711][    C1] CPU1: stopping
[14037.477718][    C1] CPU: 1 PID: 0 Comm: swapper/1 Not tainted 5.10.110 #1690
[14037.477720][    C1] Hardware name: Rockchip RK3588 EVB1 LP4 V10 Board (DT)
[14037.477722][    C1] Call trace:
[14037.477729][    C1]  dump_backtrace+0x0/0x1a8
[14037.477734][    C1]  show_stack+0x18/0x28
[14037.477739][    C7] CPU7: stopping
[14037.477744][    C1]  dump_stack_lvl+0xcc/0xf4
[14037.477749][    C6] CPU6: stopping
[14037.477753][    C1]  dump_stack+0x18/0x58
[14037.477760][    C1]  local_cpu_stop+0x60/0x70
[14037.477764][    C1]  ipi_handler+0x1a4/0x310
[14037.477769][    C4] CPU4: stopping
[14037.477778][    C1]  handle_percpu_devid_fasteoi_ipi+0x7c/0x1c8
```

假设addr是您程序重映射的PCIe设备BAR地址，您的程序对这个地址进行四个连续地址的赋值操作，请在代码点1、2、3处任意一点加入一个内存屏障 barrier();

```
/* 若在用户态使用，需要添加如下barrier的实现 */
# define barrier() __asm__ __volatile__("": : :"memory")

*((u32 *)((void *)addr + 0x0)) = 0x11111111;
/* 代码点 1 */
*((u32 *)((void *)addr + 0x4)) = 0x44444444;
/* 代码点 2 */
*((u32 *)((void *)addr + 0x8)) = 0x88888888;
/* 代码点 3 */
*((u32 *)((void *)addr + 0xc)) = 0xcccccccc;
```

3. 针对所有的PCIe设备的BAR地址进行访问，总的来说需要严格按照两个条件进行约束

- 禁止非偶数对齐地址
- 访问地址必须是访问长度的倍数，例如长度是DWord的访问，则地址不能word对齐，必须至少是DWord或者QWord对齐

还要注意编译器可能优化代码，所以如果用户态程序要求使用 -O0 编译优化等级以避免读写命令被优化。如果是链接库，也要求关闭LTO(Link-Time Optimization)，一般是查找Makefile的`-flto`标记。

2.2.9 PCIe转USB设备驱动(xhci)加载异常

部分市售PCIe转USB芯片，如VL805，在链路建立之后，设备驱动加载异常。主要异常点就是等待xHCI芯片复位没有完成，大概率是转接芯片的固件需要升级。可先对接PC平台测试，若确定需要升级固件可联系供应商。

```
[ 6.289987] pci 0000:01:00.0: xHCI HW not ready after 5 sec (HC bug?) status =
0x811
[ 6.531098] xhci_hcd 0000:01:00.0: xHCI Host Controller
[ 6.531803] xhci_hcd 0000:01:00.0: new USB bus registered, assigned bus number 3
[ 16.532539] xhci_hcd 0000:01:00.0: can't setup: -110
[ 16.533033] xhci_hcd 0000:01:00.0: USB bus 3 deregistered
[ 16.533712] xhci_hcd 0000:01:00.0: init 0000:01:00.0 fail, -110
[ 16.534281] xhci_hcd: probe of 0000:01:00.0 failed with error -110
```

若仍无法解决，可以尝试下列补丁drivers/usb/host/pci-quirks.c

```
diff --git a/drivers/usb/host/pci-quirks.c b/drivers/usb/host/pci-quirks.c
index 3ea435c..cca536d 100644
--- a/drivers/usb/host/pci-quirks.c
+++ b/drivers/usb/host/pci-quirks.c
@@ -1085,8 +1085,11 @@ static void quirk_usb_early_handoff(struct pci_dev *pdev)
    /* Skip Netlogic mips SoC's internal PCI USB controller.
     * This device does not need/support EHCI/OHCI handoff
     */
-   if (pdev->vendor == 0x184e) /* vendor Netlogic */
+   if ((pdev->vendor == 0x184e) ||
+       (pdev->vendor == PCI_VENDOR_ID_VIA && pdev->device == 0x3483)) {
+       /* 以VL805为例，其他芯片请填写正确的厂商ID和设备ID */
+       dev_warn(&pdev->dev, "bypass xhci quirk for VL805\n");
        return;
+   }
    if (pdev->class != PCI_CLASS_SERIAL_USB_UHCI &&
            pdev->class != PCI_CLASS_SERIAL_USB_OHCI &&
            pdev->class != PCI_CLASS_SERIAL_USB_EHCI &&
```

2.3 中断问题

2.3.1 设备分配到legacy中断号为0

```
0002:21:00.0 Serial controller: Device 1c00:3853 (rev 10) (prog-if 05 [16850])
        Subsystem: Device 1c00:3853
        Control: I/O- Mem- BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr-
Stepping- SERR- FastB2B- DisINTx-
        Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort-
<MAbort- >SERR- <PERR- INTx-
        Interrupt: pin A routed to IRQ 0
        Region 0: I/O ports at 1000 [disabled] [size=256]
        Region 1: Memory at 380900000 (32-bit, prefetchable) [disabled]
[size=32K]
        Region 2: I/O ports at 100100 [disabled] [size=4]
        [virtual] Expansion ROM at 380908000 [disabled] [size=32K]
        Capabilities: [60] Power Management version 3
```

可以看到pin A分配到的legacy中断号为0，实际是默认未分配状态。原理上，不论内核cmdline中是否禁用msi中断，PCIe协议栈都会在pci bus driver加载的时候调用pci_assign_irq函数读取外设配置空间的0x3d寄存器，确定pin的数值。针对所读取的pin数值进行映射并分配映射后的虚拟中断号，写入外设配置空间的0x3c寄存器。所以一般不会出现查询不到分配legacy中断号的情况。此情况下，外设将无法正常使用l发出legacy类型中断，影响设备驱动的正常执行。目前有发现部分客户从非Linux平台移植PCIe设备驱动到Linux平台上，没有适配好pci bus driver模型，导致pci_assign_irq函数未被调用，导致legacy中断未分配。

2.3.2 MSI/MSI-X无法使用

在移植外设驱动的开发过程中(主要指的是WiFi)，认为主机端的function driver因无法使用MSI或者MSI-X中断而导致流程不正常，按如下流程进行排查

- 确认前述menuconfig 中提到的配置，尤其是MSI相关配置是否都有正确勾选
- 确认rk3568.dtsi中，its节点是否被设置为disabled
- 执行lspci -vvv，查看对应设备的MSI或者MSI-X是否有支持并被使能。以此设备为例，其上报的capabilities显示其支持32个64 bit MSI，目前仅使用1个，但是 Enable-表示未使能。若正确使能应该看到Enable+，且Address应该能看到类似为0x00000000fd4400XX的地址。此情况一般是设备驱动还未加载或者加载时申请MSI或者MSI-X失败导致，请参考其他驱动，使用pci_alloc_irq_vectors等函数进行申请，详情可结合其他成熟的PCIe外设驱动做法以及参考内核中的Documentation/PCI/MSI-HOWTO.txt文档进行编写和排查异常。

```
Capabilities: [58] MSI: Enable- Count=1/32 Maskable- 64bit+
                Address: 0000000000000000  Data: 0000
```

- 如果MSI或者MSI-X有正确申请，可用如下命令导出中断计数，查看是否正常： cat /proc/interrupts 。在其中找到对应驱动申请的ITS-MSI中断(依据最后一列申请者驱动名称，例如此处为xhci_hcd驱动申请了这些MSI中断)。理论上每一笔的通信传输都会增加ITS，如果设备没有通信或者通信不正常，就会看到中断计数为0，或者有数值但发起通信后不再增加中断计数的情况。

```
229:  0  0  0  0  0  0 ITS-MSI 524288 Edge xhci_hcd
```

- 如果是概率性事件导致function driver无法收到MSI或者MSI-X中断，可以进行如下尝试。首先执行 cat /proc/interrupts 查看相应中断号，以上述229为例，将中断迁移导其他CPU测试。例如切换导CPU2，则使用命令echo 2 > /proc/irq/229/smp_affinity_list 。
- 使用协议分析仪器抓取协议信号，查看流程中外设是否有概率性没有向主机发送MSI或者MSI-X中断，而导致的异常。需注意，目前协议分析仪一般都难以支持焊贴设备的信号采集，需向设备vendor购买金手指的板卡，在我司EVB上进行测试和信号采集。另需注意我司EVB仅支持标准接口

的金手指板卡，若待测设备为M.2接口的设备(常见key A, key B, key M, key B+M, key E 四种类型)，请参考附录中"M.2 硬件接口"章节所介绍的关于不同key的外观尺寸，采购使用对应型号的转接板。

2.4 休眠唤醒问题

2.4.1 PCIe 3.0接口休眠唤醒时系统异常

休眠唤醒测试如见下列log，原因是休眠时候关闭3.3v电源时导致了时钟晶振的电源异常。请从三个方面着手：

- dts中vpcie3v3-supply的电源配置，是否电源的max和min等配置不合理，导致电源操作异常
- 测量时钟晶振，是否在休眠前提前关闭了，或者休眠失败后就没有再次开启，亦或者掉电不充分导致时钟晶振芯片异常
- 将3.3v电源和晶振的供电飞线改为外部供电，排除异常

```
[   17.406781] PM: suspend entry (deep)
[   17.406839] PM: Syncing filesystems ... done.
[   17.471710] Freezing user space processes ... (elapsed 0.002 seconds) done.
[   17.474337] OOM killer disabled.
[   17.474343] Freezing remaining freezable tasks ... (elapsed 0.001 seconds)
done.
[   17.476200] Suspending console(s) (use no_console_suspend to debug)
[   17.479152] android_work: sent uevent USB_STATE=DISCONNECTED
[   17.480290] [WLAN_RFKILL]: Enter rfkill_wlan_suspend
[   17.501382] rk-pcie 3c0000000.pcie: fail to set vpcie3v3 regulator
[   17.501406] dpm_run_callback(): genpd_suspend_noirq+0x0/0x18 returns -22
[   17.501418] PM: Device 3c0000000.pcie failed to suspend noirq: error -22
[   38.506580] rcu: INFO: rcu_preempt detected stalls on CPUs/tasks:
[   38.506601] rcu:  1-...0: (1 GPs behind) idle=25a/1/0x4000000000000000
softirq=4657/4657 fqs=2100
[   38.506604] rcu:  (detected by 0, t=6302 jiffies, g=4609, q=17)
[   38.506613] Task dump for CPU 1:
[   38.506617] kworker/u8:4    R  running task        0  1380      2 0x0000002a
[   38.506642] Workqueue: events_unbound async_run_entry_fn
[   38.506647] Call trace:
[   38.506657]  __switch_to+0xe4/0x138
[   38.506667]  pci_pm_resume_noirq+0x0/0x120
[  101.523233] rcu: INFO: rcu_preempt detected stalls on CPUs/tasks:
[  101.523250] rcu:  1-...0: (1 GPs behind) idle=25a/1/0x4000000000000000
softirq=4657/4657 fqs=8402
[  101.523253] rcu:  (detected by 0, t=25207 jiffies, g=4609, q=17)
[  101.523260] Task dump for CPU 1:
[  101.523264] kworker/u8:4    R  running task        0  1380      2 0x0000002a
[  101.523284] Workqueue: events_unbound async_run_entry_fn
[  101.523288] Call trace:
[  101.523297]  __switch_to+0xe4/0x138
[  101.523307]  pci_pm_resume_noirq+0x0/0x120
```

2.4.2 PCIe设备切换PM模式时模块异常

目前有发现个别Wi-Fi模块休眠唤醒的时候，协议栈无法将其设置从D3 hot切换为D0状态，导致休眠唤醒流程失败。由于考虑到PCIe 控制器驱动会强制链路进入L2并复位，唤醒后链路和设备能回到L0和D0状态，所以协议栈是否能成功将模块设置进入D0状态并不影响唤醒后的使用。因此可以使用所提供的补丁依次尝试进行修复。

```
[ 848.869840] PM: Some devices failed to suspend, or early wake event detected
[ 848.871600] rtl88x2ce 0000:01:00.0: Refused to change power state, currently
in D3
[ 848.946128] rtl88x2ce 0000:01:00.0: Refused to change power state, currently
in D3
[ 848.962770] rtl88x2ce 0000:01:00.0: Refused to change power state, currently
in D3
```

```
--- a/drivers/pci/quirks.c
+++ b/drivers/pci/quirks.c
@@ -1859,6 +1865,15 @@ static void quirk_d3hot_delay(struct pci_dev *dev,
unsigned int delay)
                  dev->d3_delay);
 }

+ /* Some devices report short delay it actually need, fix them! */
+static void quirk_wifi_pm(struct pci_dev *dev)
+{
+       /* Ask your vendor how long it takes for D3 to D0 !!! */
+       if (dev->vendor == PCI_VENDOR_ID_FOO && dev->device ==
PCI_DEVICE_ID_BAR)
+               quirk_d3hot_delay(dev, 200); /* e.g 200ms, can be more for test
*/
+}
+
+DECLARE_PCI_FIXUP_FINAL(PCI_VENDOR_ID_FOO, PCI_DEVICE_ID_BAR, quirk_wifi_pm);
+/* Please Add your own PID and VID */
```

```
--- a/drivers/pci/quirks.c
+++ b/drivers/pci/quirks.c
@@ -1334,6 +1334,12 @@ DECLARE_PCI_FIXUP_CLASS_EARLY(PCI_VENDOR_ID_AL,
PCI_ANY_ID,
 DECLARE_PCI_FIXUP_CLASS_EARLY(PCI_VENDOR_ID_VIA, PCI_ANY_ID,
                               PCI_CLASS_STORAGE_IDE, 8, quirk_no_ata_d3);

+/* Some Wireless devices cannot transit from D3 to D0 by writing PCI_PM_CTRL */
+DECLARE_PCI_FIXUP_CLASS_EARLY(PCI_VENDOR_ID_FOO, PCI_DEVICE_ID_BAR,
+                             PCI_CLASS_NOT_DEFINED, 8, quirk_no_ata_d3);
+/* Please Add your own PID and VID */
```

```
--- a/drivers/pci/pci.c
+++ b/drivers/pci/pci.c
@@ -818,7 +818,7 @@ static int pci_raw_set_power_state(struct pci_dev *dev,
pci_power_t state)
        if (!dev->pm_cap)
            return -EIO;
+       /* Block all requests to D3hot */
+       if (state == PCI_D3hot)
+           return 0;
```

2.4.3 PCIe 外设休眠唤醒时模块无法连接或者工作异常

原则上PCIe外设在休眠时候需要进行断电。如果由于硬件上的限制导致掉电过程过于缓慢，那么如果快速唤醒就很容易导致外设掉电不充分又被上电，引起模块工作异常。此问题常见但不限于NVMe模块，由于NVMe为了防止异常掉电而在电源增加了一些大的耦合电容，而显得问题相对突出。可以打上如下风险测试补丁，若存在上述问题，一般可以快速复现。

```
--- a/kernel/cpu.c
+++ b/kernel/cpu.c
@@ -1724,11 +1724,11 @@ int freeze_secondary_cpus(int primary)
                if (cpu == primary)
                        continue;

-               if (pm_wakeup_pending()) {
+               //if (pm_wakeup_pending()) {
                        pr_info("Wakeup pending. Abort CPU freeze\n");
                        error = -EBUSY;
                        break;
-               }
+               //}

                trace_suspend_resume(TPS("CPU_OFF"), cpu, true);
                error = _cpu_down(cpu, 1, CPUHP_OFFLINE);
```

此类问题的处理需要从两方面着手解决：

- 硬件上修改PCIe模块的放电电路，保证合理的放电时长，给出优化后的最长放电时长t。
- 软件上增加外设电源上下电的延时，确保唤醒系统时再次上电之前经过t的时间，范例如下：

```
--- a/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3568-evb1-ddr4-v10.dtsi
@@ -63,6 +63,7 @@
                enable-active-high;
                gpio = <&gpio0 RK_PD4 GPIO_ACTIVE_HIGH>;
                startup-delay-us = <5000>;
+               off-on-delay-us = <500000>; //硬件给出的最大时长t
                vin-supply = <&dc_12v>;
        };
```

2.5 稳定性问题

2.5.1 NVMe 设备长期工作状态下出现异常

```
[186825.261896] nvme nvme0: Abort status: 0x0
[187047.435152] nvme nvme0: Abort status: 0x0
[187052.740828] nvme nvme0: I/O 256 QID 1 timeout, aborting
[187082.743811] nvme nvme0: I/O 256 QID 1 timeout, reset controller
[187092.649150] nvme nvme0: Abort status: 0x0
[187389.590726] nvme nvme0: I/O 709 QID 2 timeout, aborting
[187396.088472] nvme nvme0: Abort status: 0x0
[187419.670448] nvme nvme0: I/O 736 QID 2 timeout, aborting
[187430.034886] nvme nvme0: Abort status: 0x0
[187449.750258] nvme nvme0: I/O 736 QID 2 timeout, reset controller
```

市售NVMe设备质量不一，相关问题建议先简单排除NVMe单体不良的可能：

- 对比多款不同品牌的NVMe设备测试情况
- 对比接入PC后拷机情况

重载测试下NVMe设备出现异常，从上述log可以看到NVMe设备返回的设备状态为0，因为如果链路异常，则返回应该是0xf，所以此时可排除链路问题。这种情况下，请重点从设备工作温度考虑。

- 首先需内核开启NVMe温控配置CONFIG_NVME_HWMON
- 其次遍历hwmon的ID以寻找NVMe注册的温控节点，以8为例`cat /sys/class/hwmon/hwmon8/name`，得到nvme字样即为nvme设备的温控节点。
- 读取NVMe各设备的温传计数`cat /sys/class/hwmon/hwmon8/*input`，观察重载测试下其温度变化，是否超过该设备额定工作温度。
- 若确定工作温度超出限制，需改善散热条件或者在用户态执行相应的硬盘温控策略。

```
[ 3836.614828] sysrq: Show Blocked State
[ 3836.614921] task:binder:306_2    state:D stack:    0 pid:  329 ppid:     1
flags:0x04000009
[ 3836.614925] Call trace:
[ 3836.614932] __switch_to+0x118/0x148
[ 3836.614936] __schedule+0x538/0x7a4
[ 3836.614938] schedule+0x9c/0xe0
[ 3836.614940] schedule_timeout+0x8c/0xf4
[ 3836.614942] io_schedule_timeout+0x48/0x6c
[ 3836.614944] wait_for_common_io+0x7c/0x100
[ 3836.614945] wait_for_completion_io_timeout+0x10/0x1c
[ 3836.614949] submit_bio_wait+0x70/0xb4
[ 3836.614952] blkdev_issue_discard+0x88/0xd8
[ 3836.614954] blk_ioctl_discard+0x100/0x108
[ 3836.614955] blkdev_common_ioctl+0x388/0x664
[ 3836.614957] blkdev_ioctl+0x16c/0x20c
[ 3836.614959] block_ioctl+0x34/0x44
[ 3836.614962] __arm64_sys_ioctl+0x90/0xc8
[ 3836.614964] el0_svc_common+0xac/0x1ac
[ 3836.614965] do_el0_svc+0x1c/0x28
[ 3836.614968] el0_svc+0x10/0x1c
[ 3836.614970] el0_sync_handler+0x68/0xac
[ 3836.614972] el0_sync+0x160/0x180
```

除`nvme nvme0: I/O 736 QID 2 timeout, aborting`之外，如果还触发如上打印。此时应分析是否在异常前进行了全盘格式化或者同步批量删除等操作，尤其是大容量NVMe在较满的状态下更易触发。解决办法如下：

```
(1) 将CONFIG_DEFAULT_HUNG_TASK_TIMEOUT配置一个稍大秒数，例如120，以规避block-mq的阻塞检测
```

```
(2) 调整NVMe协议栈关于admin queue和io queue的软件超时时间
--- a/drivers/nvme/host/core.c
+++ b/drivers/nvme/host/core.c
@@ -41,12 +41,12 @@ struct nvme_ns_info {
        bool is_removed;
 };

-unsigned int admin_timeout = 60;
+unsigned int admin_timeout = 120;
 module_param(admin_timeout, uint, 0644);
 MODULE_PARM_DESC(admin_timeout, "timeout in seconds for admin commands");
 EXPORT_SYMBOL_GPL(admin_timeout);

-unsigned int nvme_io_timeout = 30;
+unsigned int nvme_io_timeout = 120;
 module_param_named(io_timeout, nvme_io_timeout, uint, 0644);
```

若出现类似如下log表明NVMe颗粒异常，无法查明原因并修复情况下可以尝试添加复位流程

```
nvme nvme0: I/O 528 QID 10 timeout, reset controller
nvme nvme0: controller is down; will reset: CSTS=0xffffffff, PCI_STATUS=0x10
```

```
--- a/drivers/nvme/host/pci.c
+++ b/drivers/nvme/host/pci.c
@@ -11,6 +11,9 @@
 #include <linux/blk-mq.h>
 #include <linux/blk-mq-pci.h>
 #include <linux/blk-integrity.h>
+#ifdef CONFIG_ARCH_ROCKCHIP
+#include <linux/aspm_ext.h>
+#endif
 #include <linux/dmi.h>
 #include <linux/init.h>
 #include <linux/interrupt.h>
@@ -132,6 +135,7 @@ struct nvme_dev {
        void __iomem *bar;
        unsigned long bar_mapped_size;
        struct work_struct remove_work;
+       struct pci_saved_state *state;
        struct mutex shutdown_lock;
        bool subsystem;
        u64 cmb_size;
@@ -1324,6 +1328,21 @@ static void nvme_warn_reset(struct nvme_dev *dev, u32
csts)
                 "Try \"nvme_core.default_ps_max_latency_us=0 pcie_aspm=off
pcie_port_pm=off\" and report a bug\n");
 }
+
+static struct pci_dev *nvme_get_rc(struct pci_dev *pdev)
+{
+       struct pci_dev *rc;
+
+       /* Walk bus to get root port */
+       rc = pdev;
+       while (rc->bus->parent) {
+               rc = rc->bus->self;
```

```
+               if (!rc)
+                       break;
+       }
+
+       return rc;
+}
+
 static enum blk_eh_timer_return nvme_timeout(struct request *req)
 {
        struct nvme_iod *iod = blk_mq_rq_to_pdu(req);
@@ -1344,6 +1363,9 @@ static enum blk_eh_timer_return nvme_timeout(struct
request *req)
         * Reset immediately if the controller is failed
         */
        if (nvme_should_reset(dev, csts)) {
+               rockchip_dw_pcie_pm_ctrl_for_user(nvme_get_rc(to_pci_dev(dev-
>dev)),
+                                                 ROCKCHIP_PCIE_PM_CTRL_RESET);
+               pci_restore_state(to_pci_dev(dev->dev));
                nvme_warn_reset(dev, csts);
                nvme_dev_disable(dev, false);
                nvme_reset_ctrl(&dev->ctrl);
@@ -1839,9 +1861,18 @@ static int nvme_pci_configure_admin_queue(struct nvme_dev
*dev)
        lo_hi_writeq(nvmeq->sq_dma_addr, dev->bar + NVME_REG_ASQ);
        lo_hi_writeq(nvmeq->cq_dma_addr, dev->bar + NVME_REG_ACQ);

+       pci_save_state(to_pci_dev(dev->dev));
+
        result = nvme_enable_ctrl(&dev->ctrl);
-       if (result)
-               return result;
+       if (result) {
+               rockchip_dw_pcie_pm_ctrl_for_user(nvme_get_rc(to_pci_dev(dev-
>dev)),
+                                                 ROCKCHIP_PCIE_PM_CTRL_RESET);
+               pci_restore_state(to_pci_dev(dev->dev));
+               /* Try again */
+               result = nvme_enable_ctrl(&dev->ctrl);
+               if (result)
+                       return result;
+       }

        nvmeq->cq_vector = 0;
        nvme_init_queue(nvmeq, 0);
@@ -3135,10 +3166,12 @@ static unsigned long check_vendor_combination_bug(struct
pci_dev *pdev)
 static void nvme_async_probe(void *data, async_cookie_t cookie)
 {
        struct nvme_dev *dev = data;
+       struct pci_dev *pdev = to_pci_dev(dev->dev);

        flush_work(&dev->ctrl.reset_work);
        flush_work(&dev->ctrl.scan_work);
        nvme_put_ctrl(&dev->ctrl);
+       dev->state = pci_store_saved_state(pdev);
 }
```

2.6 其他问题

2.6.1 PCIe转SATA设备盘号不固定

因为驱动的初始化是采用线程化，所以多个PCIe控制器下外接的转接SATA芯片的初始化顺序不固定。实际上，每个转接SATA芯片下的多个硬盘口号是固定的，但是多个SATA芯片之间的硬盘口号是不固定的。例如某次启动，转接芯片1下的四个硬盘分别为sda~sdd, 转接芯片2下的四个硬盘分别为sde~sdh；下一次启动可能变成转接芯片1下的四个硬盘分别为sde~sdh, 转接芯片2下的四个硬盘分别为sda~sdd的情况。为了完全固定这些硬盘的顺序，可以采用应用层遍历硬盘路径，按照控制器地址(例如fe160000.pcie)来固定转接芯片，进而创建软连接来固定顺序。如果想简化应用层，也可以采用关闭线程初始化，将CONFIG_PCIE_RK_THREADED_INIT去除即可。

```
ls -al /sys/block/sd*

rwxrwxrwx    1 root     root             0 Jan  1 00:00 /sys/block/sda ->
../devices/platform/fe160000.pcie/pci0001:10/0001:10:00.0/0001:11:00.0/ata2/host
1/target1:0:0/1:0:0:0/block/sda

lrwxrwxrwx    1 root     root             0 Jan  1 00:00 /sys/block/sdb ->
../devices/platform/fe160000.pcie/pci0001:10/0001:10:00.0/0001:11:00.0/ata3/host
2/target2:0:0/2:0:0:0/block/sdb

lrwxrwxrwx    1 root     root             0 Jan  1 00:03 /sys/block/sdc ->
../devices/platform/fe160000.pcie/pci0001:10/0001:10:00.0/0001:11:00.0/ata4/host
3/target3:0:0/3:0:0:0/block/sdc

lrwxrwxrwx    1 root     root             0 Jan  1 00:00 /sys/block/sdd ->
../devices/platform/fe160000.pcie/pci0001:10/0001:10:00.0/0001:11:00.0/ata5/host
4/target4:0:0/4:0:0:0/block/sdd
```

2.6.2 GKI模式下PCIe外设工作异常

目前6.1(含)之前的GKI版本，PCIe外设只能使用Legacy中断，请执行`cat /proc/cmdline`，检索是否已经设置了`nomsi`标志。另外，由于PCIe2.0口与USB3或者SATA复用，如果两者KO都使能并加载，会导致异常，检索log中可看到类似关键字`naneng-combphy fee20000.phy: phy type select 2 overwriting type 4`。

3. 附录

3.1 LTSSM状态机

状态描述符	状态代码 (6'h)	备注
S_DETECT_QUIET	0x00	未检测到外设RX端接电阻，设备未正常工作
S_DETECT_ACTIVE	0x01	-
S_POLL_ACTIVE	0x02	-
S_POLL_COMPLIANCE	0x03	兼容性测试模式。非测试模式下因信号异常原因错误进入
S_POLL_CONFIG	0x04	-
S_PRE_DETECT_QUIET	0x05	-
S_DETECT_WAIT	0x06	-
S_CFG_LINKWD_START	0x07	-
S_CFG_LINKWD_ACEPT	0x08	Lane 顺序检测过程
S_CFG_LANENUM_WAIT	0x09	-
S_CFG_LANENUM_ACEPT	0x0a	-
S_CFG_COMPLETE	0x0b	物理层检测完毕
S_CFG_IDLE	0x0c	-
S_RCVRY_LOCK	0x0d	速率切换过程
S_RCVRY_SPEED	0x0e	-
S_RCVRY_RCVRCFG	0x0f	-
S_RCVRY_IDLE	0x10	-
S_RCVRY_EQ0	0x20	-
S_RCVRY_EQ1	0x21	-
S_RCVRY_EQ2	0x22	-
S_RCVRY_EQ3	0x23	-
S_L0	0x11	链路工作状态L0
S_L0S	0x12	-
S_L123_SEND_EIDLE	0x13	-
S_L1_IDLE	0x14	-
S_L2_IDLE	0x15	-
S_L2_WAKE	0x16	-
S_DISABLED_ENTRY	0x17	-
S_DISABLED_IDLE	0x18	-

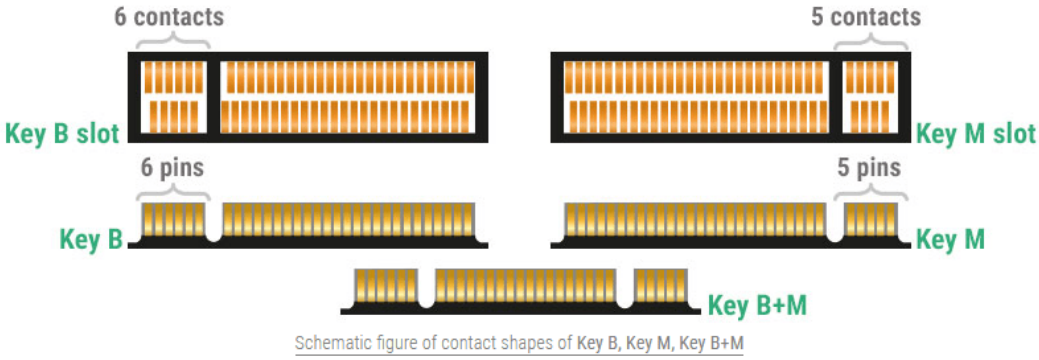
状态描述符	状态代码 (6'h)	备注
S_DISABLED	0x19	-
S_LPBK_ENTRY	0x1a	-
S_LPBK_ACTIVE	0x1b	loopback测试模式，一般在测试环境下会进入
S_LPBK_EXIT	0x1c	-
S_LPBK_EXIT_TIMEOUT	0x1d	-
S_HOT_RESET_ENTRY	0x1e	外设主动发起hot reset流程
S_HOT_RESET	0x1f	

3.2 PCIe地址空间配置详述

针对DTS中PCIe地址空间的详细配置，可参考如下链接进行解读或者修改：https://elinux.org/Device_Tree_Usage#PCI_Host_Bridge

3.3 M.2接口硬件

Key B+M的金手指可以使用key B或者key M的转接板slot，key A+E的金手指可以使用key A或者key E的转接板slot。其余类型的金手指板请严格按照接口选择对应型号的转接板slot。



Schematic figure of contact shapes of Key B, Key M, Key B+M

